

Quantum Annealing for Earth–Moon Low-Thrust Trajectory Optimization

Gino Luciano Moretta

Sapienza Università di Roma, supervisor Prof. A. Paolozzi, E. Ortore

Last updated: April 2026

Contents

1	Project overview	2
2	Units and the planar CR3BP	3
2.1	Nondimensionalization	3
2.2	Equations of motion	3
2.3	The Jacobi integral	4
2.4	Lagrange points	4
2.5	Linearization of (1)	4
3	Optimal control formulation	5
3.1	The problem	5
3.2	Hamiltonian and Pontryagin	5
3.3	Linearized costate equations	6
4	Indirect TFC+ELM transcription	6
4.1	Theory of Functional Connections in one picture	6
4.2	Extreme Learning Machine basis	7
4.3	Constrained expression assembly	7
4.4	The linear residual system	8
4.5	Number of unknowns and size of the linear system	8
5	Linear LSQ $\rightarrow$ QUBO pipeline	8
5.1	Step 1 — Tikhonov regularization	9
5.2	Step 2 — Truncated SVD	9
5.3	Step 3 — Per-variable adaptive encoding	9
5.4	Step 4 — QUBO assembly	10
5.5	Decoding and warm starts	10
5.6	Default hyperparameters	10
6	Sequential linearization outer loop	11
6.1	Pass-2 Armijo line search	11
7	Classical reference via direct collocation	13
7.1	Hermite–Simpson direct collocation	13
7.2	Resampling onto the TFC+ELM grid	14
7.3	Example trajectories: a two-phase Earth–Moon mission	14
8	Code organization	16
8.1	Running the tests	16
8.2	Key modules at a glance	16

9	Convergence investigation and pivot	17
9.1	Armijo line search failure	17
9.2	QUBO quantization error	17
9.3	Why polishing does not save the approach	18
9.4	Conclusion: the continuous-lstsq QUBO is the wrong problem	18
10	On/off thrust scheduling via QUBO	18
10.1	Problem setup	18
10.2	Linearization via the state transition matrix	18
10.3	QUBO formulation	19
10.4	Solvers	19
10.5	Iterative re-linearization	20
10.6	Multi-direction scheduling	20
11	Scheduling results	21
11.1	Validation: SA matches brute force	21
11.2	Scaling analysis	21
11.3	Where quantum advantage emerges	22
11.4	Multi-direction results	22
11.5	Linearization-gap and iterative correction	22
11.6	Second application: lunar-orbit station-keeping	23
11.7	Classical MILP baseline	23
12	Multi-arc capture and end-to-end mission	24
12.1	Sliding-window QUBO for lunar capture (Phase 3)	24
12.2	End-to-end mission: composing Phases 1, 2 and 3	25
12.3	Connection to primer-vector theory	26
12.4	Comparison with state-of-the-art Earth–Moon transfers	27
13	Discussion: why the project changed direction	28
14	Glossary of symbols	29

1 Project overview

Research goal. Extend the pseudospectral quantum-annealing transcription framework of De Grossi, Carbone et al. (*Astrodynamics* 9:195–215, 2025) [1] from Earth–Mars interplanetary low-thrust transfers to **Earth–Moon** low-thrust transfers in the planar circular restricted three-body problem (CR3BP), and evaluate the resulting problem on D-Wave’s **Advantage 2** system (and its nonlinear hybrid solver) as part of a six-month research project in aerospace engineering at Sapienza.

Companion paper. The scientific account of the on/off scheduling formulation, the iterative re-linearisation methodology, the classical mixed-integer-linear-programming baseline, and the experimental results on the cislunar injection-correction scenario is given in the companion paper, *Iterative Re-Linearisation for a Naturally-Binary QUBO Formulation of Low-Thrust Cislunar Trajectory Scheduling* (paper/paper.tex). This manual is the technical reference for the implementation: it covers the full codebase end-to-end, including the earlier continuous-coefficient QUBO that the project pivoted away from, and is intended for developers and reviewers reading the source.

Why the Earth–Moon case is harder. Carbone et al. worked in an *inertial two-body* (Keplerian + small perturbations) setting where the dynamics are essentially linear around the reference heliocentric arc. The Earth–Moon CR3BP in the rotating (**synodic**) frame is *genuinely nonlinear*: the gradient of the pseudo-potential contains $1/r^2$ terms that blow up near each primary. A single-shot least-squares $\rightarrow$ QUBO pipeline cannot accommodate this directly, so we layer a **sequential linearization outer loop** on top of the QUBO solver, at each iteration the nonlinear dynamics are linearized about the current best trajectory, a new linear system is built, and the QUBO solver is called again.

Two transcription philosophies. There are two principled ways to convert an optimal control problem into a linear residual system that can be fed into a QUBO:

- **Direct:** discretize state and control at a set of collocation nodes (e.g. Legendre–Gauss–Lobatto), and write the dynamics constraints as algebraic residuals. Carbone et al. use this approach with LGL nodes and a Lagrange-polynomial differentiation matrix.
- **Indirect:** apply Pontryagin’s Minimum Principle analytically to eliminate the control (for an energy-minimizing cost, $\mathbf{u}^* = -\boldsymbol{\lambda}_v$), obtain the two-point boundary value problem for state *and* costate, and then discretize by a trial-function expansion. Gino’s existing v2 rendezvous code uses this path with **Theory of Functional Connections (TFC)** switching functions plus an **Extreme Learning Machine (ELM)** basis for the free part of the solution.

This project will implement **both** and compare them on the same Earth–Moon benchmark. The indirect TFC+ELM pipeline is implemented first because it reuses almost all of Gino’s validated rendezvous machinery; the direct LGL pipeline is implemented second as a baseline.

2 Units and the planar CR3BP

2.1 Nondimensionalization

We work in **canonical CR3BP units**. Let m_1 (Earth) and m_2 (Moon) be the two primaries, a the semi-major axis of their mutual orbit, and n the mean motion. Define:

- **Length unit** $L = a \approx 384\,400$ km.
- **Time unit** $T = 1/n$; one nondimensional time unit corresponds to roughly 4.343 days $\approx 3.757 \times 10^5$ s.
- **Mass unit** $m_1 + m_2$, so the mass parameter is

$$\mu = \frac{m_2}{m_1 + m_2} \approx 0.012\,150\,583\,925\,359 \quad (\text{Earth–Moon}).$$

With this scaling, the gravitational constant times total mass is unity, and the angular velocity of the rotating (synodic) frame is $\omega = 1$. The primaries become *stationary* on the x -axis:

$$\mathbf{r}_{\oplus}^{\text{syn}} = (-\mu, 0), \quad \mathbf{r}_{\text{Moon}}^{\text{syn}} = (1 - \mu, 0).$$

Remark. This is why the CR3BP is such a clean playground for mission design: in the synodic frame, geometry is frozen, and the dynamics become a time-invariant vector field on a 4D phase space.

2.2 Equations of motion

Let $\mathbf{x} = (x, y)$ be the synodic position and $\mathbf{u} = (u_x, u_y)$ the thrust-acceleration control (low-thrust is modelled as a continuous acceleration, not an impulsive Δv). The planar CR3BP with control reads

$$\begin{cases} \ddot{x} = -2\dot{y} + \Omega_x(x, y) + u_x, \\ \ddot{y} = 2\dot{x} + \Omega_y(x, y) + u_y, \end{cases} \quad (1)$$

where the pseudo-potential Ω and its gradient are

$$\Omega(x, y) = \frac{1}{2}(x^2 + y^2) + \frac{1 - \mu}{r_1} + \frac{\mu}{r_2} + \frac{1}{2}\mu(1 - \mu), \quad (2)$$

$$\Omega_x(x, y) = x - \frac{(1 - \mu)(x + \mu)}{r_1^3} - \frac{\mu(x - 1 + \mu)}{r_2^3}, \quad (3)$$

$$\Omega_y(x, y) = y - \frac{(1 - \mu)y}{r_1^3} - \frac{\mu y}{r_2^3}, \quad (4)$$

with distances from the two primaries

$$r_1 = \sqrt{(x + \mu)^2 + y^2}, \quad r_2 = \sqrt{(x - 1 + \mu)^2 + y^2}.$$

The two $2\dot{y}$ and $-2\dot{x}$ terms are the **Coriolis coupling** from the rotating frame; the centrifugal contribution has been absorbed into the $\frac{1}{2}(x^2 + y^2)$ part of Ω .

In first-order form with state $\mathbf{x} = (x, y, v_x, v_y)$:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{B}\mathbf{u}, \quad \mathbf{B} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{f}(\mathbf{x}) = \begin{pmatrix} v_x \\ v_y \\ 2v_y + \Omega_x \\ -2v_x + \Omega_y \end{pmatrix}. \quad (5)$$

This is the form consumed by `PlanarCR3BP.rhs` in the code.

2.3 The Jacobi integral

Along any unforced ($\mathbf{u} = \mathbf{0}$) trajectory, the CR3BP conserves the **Jacobi integral**

$$C_J(\mathbf{x}) = 2\Omega(x, y) - v_x^2 - v_y^2. \quad (6)$$

This is the CR3BP's analogue of mechanical energy (in the synodic frame) and is the single most useful sanity check for any integrator or transcription: a pure-drift propagator that lets C_J wander immediately gives away a bug. The code verifies C_J conservation to $< 10^{-8}$ over 10^3 RK4 steps as a unit test.

2.4 Lagrange points

Equation (1) has **five** equilibria in the rotating frame — the five Lagrange points L_1, L_2, L_3 (collinear, on the x -axis) and L_4, L_5 (triangular, on the equilateral triangles with the primaries). For Earth–Moon:

Point	Approx. x	Approx. y	Stability (linear)
L_1	0.8369	0	saddle-center
L_2	1.1557	0	saddle-center
L_3	-1.0051	0	saddle-center
L_4	$0.5 - \mu$	$+\sqrt{3}/2$	stable ($\mu < 0.0385$)
L_5	$0.5 - \mu$	$-\sqrt{3}/2$	stable

The triangular points L_4 and L_5 sit at the apex of the equilateral triangles $\oplus$ -Moon- $L_{4/5}$, i.e. at $r_1 = r_2 = 1$, and — because of this and a minor bookkeeping cancellation — the Jacobi integral there is *exactly* $C_J = 3$ independently of μ .

2.5 Linearization of (1)

The QUBO pipeline downstream is fundamentally a *linear* least-squares engine; it needs the dynamics to be linear in the unknowns. For the CR3BP we therefore linearize (1) about a **nominal trajectory** $\bar{\mathbf{x}}(t) = (\bar{x}(t), \bar{y}(t), \bar{v}_x(t), \bar{v}_y(t))$ and solve for the *deviation* $\delta\mathbf{x} = \mathbf{x} - \bar{\mathbf{x}}$.

Taylor-expanding Ω_x and Ω_y :

$$\begin{aligned} \Omega_x(x, y) &\approx \Omega_x(\bar{x}, \bar{y}) + \Omega_{xx}(\bar{x}, \bar{y}) \delta x + \Omega_{xy}(\bar{x}, \bar{y}) \delta y, \\ \Omega_y(x, y) &\approx \Omega_y(\bar{x}, \bar{y}) + \Omega_{xy}(\bar{x}, \bar{y}) \delta x + \Omega_{yy}(\bar{x}, \bar{y}) \delta y, \end{aligned} \quad (7)$$

where the Hessian components are

$$\Omega_{xx} = 1 - \frac{1 - \mu}{r_1^3} - \frac{\mu}{r_2^3} + \frac{3(1 - \mu)(x + \mu)^2}{r_1^5} + \frac{3\mu(x - 1 + \mu)^2}{r_2^5}, \quad (8)$$

$$\Omega_{yy} = 1 - \frac{1 - \mu}{r_1^3} - \frac{\mu}{r_2^3} + \frac{3(1 - \mu)y^2}{r_1^5} + \frac{3\mu y^2}{r_2^5}, \quad (9)$$

$$\Omega_{xy} = \frac{3(1 - \mu)(x + \mu)y}{r_1^5} + \frac{3\mu(x - 1 + \mu)y}{r_2^5}. \quad (10)$$

These come from `PlanarCR3BP.jacobian_state`, which is tested against central finite differences across six random seeds.

Substituting the expansion into (1) and subtracting the nominal equation yields a linear time-varying (LTV) system for $\delta\mathbf{x}$:

$$\begin{cases} \delta\ddot{x} = 2\delta\dot{y} + \Omega_{xx}(t)\delta x + \Omega_{xy}(t)\delta y + \delta u_x, \\ \delta\ddot{y} = -2\delta\dot{x} + \Omega_{xy}(t)\delta x + \Omega_{yy}(t)\delta y + \delta u_y, \end{cases} \quad (11)$$

where the coefficients $\Omega_{xx}(t) \equiv \Omega_{xx}(\bar{x}(t), \bar{y}(t))$ etc. are *known* functions of time once the nominal is fixed.

Remark. Equation (11) should look familiar: with the nominal frozen at L_4 and Earth–Moon μ , it reduces exactly to the classical L4 libration linear analysis. With the nominal frozen at the Earth, it reduces to Clohessy–Wiltshire-like behaviour (up to a different reference frame). Both are physically meaningful sanity-check regimes for the transcription.

3 Optimal control formulation

3.1 The problem

We want to steer the spacecraft from an initial **low Earth orbit (LEO)** state to a final **low lunar orbit (LLO)** state over a fixed time of flight T :

$$\min_{\mathbf{u}(\cdot)} J = \frac{1}{2} \int_0^T \|\mathbf{u}(t)\|^2 dt \quad (12)$$

subject to the CR3BP dynamics (5) and the boundary conditions

$$\mathbf{r}(0) = \mathbf{r}_0, \quad \mathbf{v}(0) = \mathbf{v}_0, \quad \mathbf{r}(T) = \mathbf{r}_f, \quad \mathbf{v}(T) = \mathbf{v}_f. \quad (13)$$

The $\frac{1}{2}\|\mathbf{u}\|^2$ cost is the standard **energy-optimal** objective for low-thrust rendezvous. It is *not* fuel-optimal ($\int \|\mathbf{u}\| dt$), but it is quadratic in the control so it stays friendly to QUBO and least-squares pipelines. Fuel-optimal cost is an explicit future extension.

3.2 Hamiltonian and Pontryagin

Introduce the costate $\boldsymbol{\lambda}(t) = (\boldsymbol{\lambda}_r, \boldsymbol{\lambda}_v)^\top$, partitioned so that $\boldsymbol{\lambda}_r$ is conjugate to $\mathbf{r}$ and $\boldsymbol{\lambda}_v$ is conjugate to $\mathbf{v}$. The control Hamiltonian is

$$\mathcal{H}(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}) = \frac{1}{2}\|\mathbf{u}\|^2 + \boldsymbol{\lambda}_r^\top \mathbf{v} + \boldsymbol{\lambda}_v^\top (\mathbf{g}(\mathbf{r}) + \mathbf{C}\mathbf{v} + \mathbf{u}), \quad (14)$$

where $\mathbf{g} = (\Omega_x, \Omega_y)$ is the pseudo-potential gradient and

$$\mathbf{C} = \begin{pmatrix} 0 & 2 \\ -2 & 0 \end{pmatrix} \quad (15)$$

is the Coriolis block (constant). Pontryagin’s Minimum Principle gives the optimality conditions:

$$\dot{\mathbf{r}} = \partial\mathcal{H}/\partial\boldsymbol{\lambda}_r = \mathbf{v}, \quad (16)$$

$$\dot{\mathbf{v}} = \partial\mathcal{H}/\partial\boldsymbol{\lambda}_v = \mathbf{g}(\mathbf{r}) + \mathbf{C}\mathbf{v} + \mathbf{u}, \quad (17)$$

$$\dot{\boldsymbol{\lambda}}_r = -\partial\mathcal{H}/\partial\mathbf{r} = -(\partial\mathbf{g}/\partial\mathbf{r})^\top \boldsymbol{\lambda}_v, \quad (18)$$

$$\dot{\boldsymbol{\lambda}}_v = -\partial\mathcal{H}/\partial\mathbf{v} = -\boldsymbol{\lambda}_r - \mathbf{C}^\top \boldsymbol{\lambda}_v, \quad (19)$$

and the optimality equation

$$0 = \partial\mathcal{H}/\partial\mathbf{u} = \mathbf{u} + \boldsymbol{\lambda}_v \quad \implies \quad \boxed{\mathbf{u}^* = -\boldsymbol{\lambda}_v}. \quad (20)$$

Consequence. The control is *eliminated analytically*. The two-point BVP for the indirect method has unknowns $(\mathbf{r}, \mathbf{v}, \boldsymbol{\lambda}_r, \boldsymbol{\lambda}_v)$ — 8 scalar functions in the planar case — with 8 boundary conditions (four from the BCs (13) on $\mathbf{r}$, four on $\mathbf{v}$). The costates are *unconstrained* at the boundaries: they adjust automatically to whatever keeps the optimality conditions consistent.

3.3 Linearized costate equations

When the state dynamics are linearized about a nominal as in (11), the costate dynamics linearize accordingly because the Hessian of Ω is what appears in the state Jacobian. Writing out all four components:

$$\dot{\lambda}_{r_x} = -(\Omega_{xx}\lambda_{v_x} + \Omega_{xy}\lambda_{v_y}), \quad (21)$$

$$\dot{\lambda}_{r_y} = -(\Omega_{xy}\lambda_{v_x} + \Omega_{yy}\lambda_{v_y}), \quad (22)$$

$$\dot{\lambda}_{v_x} = -\lambda_{r_x} + 2\lambda_{v_y}, \quad (23)$$

$$\dot{\lambda}_{v_y} = -\lambda_{r_y} - 2\lambda_{v_x}, \quad (24)$$

where all Ω Hessian coefficients are evaluated at the nominal trajectory (so they are *known functions of time*). These are the equations the transcription will discretize.

4 Indirect TFC+ELM transcription

4.1 Theory of Functional Connections in one picture

The **Theory of Functional Connections** (TFC), due to Mortari [4], gives a way to write down a general functional form that *automatically satisfies any set of linear boundary conditions*. The basic idea:

Instead of enforcing boundary conditions as separate equations, build them into the basis. The basis becomes a family of “constrained expressions” indexed by free parameters $\mathbf{x}$, and every choice of $\mathbf{x}$ lands on a function that already meets the BCs by construction.

For a second-order ODE on $[0, T]$ with four BCs on position and velocity,

$$x(0) = x_0, \quad x(T) = x_f, \quad \dot{x}(0) = v_0, \quad \dot{x}(T) = v_f, \quad (25)$$

the constrained expression takes the form

$$x(t) = C(t) + F(t, \mathbf{x}), \quad (26)$$

where $C(t)$ is a particular solution hitting the BCs and $F(t, \mathbf{x})$ is the free part *annihilated by all four BC projections*. TFC provides the following standard choice in terms of switching functions $\omega_1, \omega_2, \omega_3, \omega_4$:

$$C(t) = \omega_1(t)x_0 + \omega_2(t)x_f + \omega_3(t)v_0 + \omega_4(t)v_f, \quad (27)$$

$$F(t, \mathbf{x}) = g(t, \mathbf{x}) - \omega_1(t)g(0, \mathbf{x}) - \omega_2(t)g(T, \mathbf{x}) - \omega_3(t)\dot{g}(0, \mathbf{x}) - \omega_4(t)\dot{g}(T, \mathbf{x}), \quad (28)$$

where $g(\cdot, \mathbf{x})$ is *any* twice-differentiable function of t parameterised by unknowns $\mathbf{x}$. The four ω_i are cubic Hermite basis polynomials on $[0, T]$ with the following “Kronecker-like” property:

	ω_1	ω_2	ω_3	ω_4
value at 0	1	0	0	0
value at T	0	1	0	0
derivative at 0	0	0	1	0
derivative at T	0	0	0	1

(29)

so that $C(0) = x_0$, $C(T) = x_f$, $\dot{C}(0) = v_0$, $\dot{C}(T) = v_f$, and F contributes 0 to all four BC projections for any $\mathbf{x}$. It follows that the full expression satisfies all four BCs *independently* of $\mathbf{x}$ — the optimizer can search freely in the space of $\mathbf{x}$ without ever violating a BC.

Explicit formulas. On the normalized interval $z \in [-1, 1]$ with $dz = 2$, the implementation uses

$$\omega_1(z) = 1 + 2\tau^3/8 - 3\tau^2/4, \quad (30)$$

$$\omega_2(z) = -2\tau^3/8 + 3\tau^2/4, \quad (31)$$

$$\omega_3(z) = \tau + \tau^3/4 - \tau^2, \quad (32)$$

$$\omega_4(z) = \tau^3/4 - \tau^2/2, \quad (33)$$

with $\tau = z - z_0 = z + 1$. These are the same Hermite basis polynomials used in Gino’s rendezvous code; a time-rescaling factor $c = (z_f - z_0)/(t_f - t_0) = 2/T$ converts derivatives with respect to z back into derivatives with respect to physical time.

4.2 Extreme Learning Machine basis

The “free” function $g(t, \mathbf{x})$ that lives inside F can be *any* twice-differentiable parameterization. TFC experiments typically use a **single-hidden-layer neural network with random, frozen input weights** — an **Extreme Learning Machine** (ELM) [5]:

$$g(z, \mathbf{x}) = \sum_{j=1}^L \xi_j \sigma(w_j z + b_j), \quad (34)$$

where σ is a smooth activation (e.g. tanh, logistic, or Gaussian), the weights w_j and biases b_j are drawn *once* from a fixed distribution (e.g. uniform on $[-3, 3]$) and *never updated*, and the output weights ξ_j are the only free parameters.

An ELM is a universal approximator: for any sufficiently smooth target function g^* and any error tolerance ε , there exists an ELM with enough neurons L that gets within ε of g^* by an appropriate choice of $\mathbf{x}$. Crucially, because the output layer is linear in $\mathbf{x}$, the LSQ problem “fit $g(\cdot, \mathbf{x})$ to residual data” is linear in $\mathbf{x}$ — exactly what we need for the downstream QUBO engine.

Why not spline or Chebyshev?

Two reasons ELM is preferred here:

1. **Random-feature universality.** There is no rigid grid and no dependence on collocation structure, you simply draw more neurons to get more capacity.
2. **Well-conditioning of the transcription matrix.** Gaussian and tanh ELM features are effectively random Fourier-like features: the linear system assembled from them typically has a smoothly decaying singular spectrum that plays well with the truncated-SVD step of the QUBO pipeline.

Derivatives of the ELM basis

At each training point z_i and each neuron j , we need three quantities: $\sigma, \sigma', \sigma''$. For $\sigma = \tanh$:

$$\sigma(wz + b) = \tanh(wz + b), \quad (35)$$

$$\sigma'(wz + b) = w(1 - \tanh^2(wz + b)), \quad (36)$$

$$\sigma''(wz + b) = -2w^2 \tanh(wz + b)(1 - \tanh^2(wz + b)). \quad (37)$$

Stacking these over n training points and L neurons gives three matrices $\mathbf{H}, \mathbf{H}', \mathbf{H}'' \in \mathbb{R}^{n \times L}$.

4.3 Constrained expression assembly

With the switching functions and the ELM basis in hand, build the following per-coordinate constrained matrices on the n training points:

$$\mathbf{F} = \mathbf{H} - \omega_1 \mathbf{H}(z_0) - \omega_2 \mathbf{H}(z_f) - \omega_3 \mathbf{H}'(z_0) - \omega_4 \mathbf{H}'(z_f), \quad (38)$$

$$\mathbf{F}' = c \left[\mathbf{H}' - \omega'_1 \mathbf{H}(z_0) - \omega'_2 \mathbf{H}(z_f) - \omega'_3 \mathbf{H}'(z_0) - \omega'_4 \mathbf{H}'(z_f) \right], \quad (39)$$

$$\mathbf{F}'' = c^2 \left[\mathbf{H}'' - \omega''_1 \mathbf{H}(z_0) - \dots \right]. \quad (40)$$

The factor $c = 2/T$ is the chain-rule conversion between z -time and physical time, and appears once per derivative order.

Note the crucial property: $\mathbf{F}$ is zero in the first and last rows by construction (the switching-function subtraction exactly cancels the value at the endpoints), and likewise $\mathbf{F}'$ is zero at the endpoints. This encodes the BC annihilation.

The BC-particular part is assembled similarly:

$$\mathbf{C}(z) = \omega_1 r_0 + \omega_2 r_f + \frac{1}{c} [\omega_3 v_0 + \omega_4 v_f], \quad (41)$$

with the $1/c$ for velocity BCs because velocities in physical time are the z -derivative times c . Evaluating $\mathbf{C}$ and its derivatives at each training point gives three vectors $\mathbf{C}, \mathbf{C}', \mathbf{C}'' \in \mathbb{R}^n$.

4.4 The linear residual system

Gather all the unknown ELM weights into a single tall vector:

$$\boldsymbol{\xi} = (\boldsymbol{\xi}_x, \boldsymbol{\xi}_y, \boldsymbol{\xi}_{\lambda_{rx}}, \boldsymbol{\xi}_{\lambda_{ry}}, \boldsymbol{\xi}_{\lambda_{vx}}, \boldsymbol{\xi}_{\lambda_{vy}}) \in \mathbb{R}^{6L}, \quad (42)$$

so there are six blocks of L weights — two for the position states (which use $\mathbf{F}$ -based constrained expressions with BCs) and four for the costates (which use $\mathbf{H}$ -based expressions *without* BC enforcement, since the costates have no fixed boundary values).

At each of the n training points, the six ODEs yield $6n$ linear residual equations. Written as a block matrix:

$$\underbrace{\begin{pmatrix} \mathbf{F}'' - \Omega_{xx}\mathbf{F} & -\Omega_{xy}\mathbf{F} & \mathbf{0} & \mathbf{0} & \mathbf{H} & \mathbf{0} \\ -\Omega_{xy}\mathbf{F} & \mathbf{F}'' - \Omega_{yy}\mathbf{F} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{H} \\ \mathbf{0} & \mathbf{0} & \mathbf{H}' & \mathbf{0} & \Omega_{xx}\mathbf{H} & \Omega_{xy}\mathbf{H} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{H}' & \Omega_{xy}\mathbf{H} & \Omega_{yy}\mathbf{H} \\ \mathbf{0} & \mathbf{0} & \mathbf{H} & \mathbf{0} & \mathbf{H}' & -2\mathbf{H} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{H} & 2\mathbf{H} & \mathbf{H}' \end{pmatrix}}_{\mathbf{A} \in \mathbb{R}^{6n \times 6L}} \boldsymbol{\xi} = \underbrace{\begin{pmatrix} -\mathbf{C}_x'' + 2\mathbf{C}_y' + \tilde{g}_x \\ -\mathbf{C}_y'' - 2\mathbf{C}_x' + \tilde{g}_y \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \end{pmatrix}}_{\mathbf{B} \in \mathbb{R}^{6n}}, \quad (43)$$

where the nominal-forcing terms that enter the first two blocks of $\mathbf{B}$ are

$$\tilde{g}_x(t) = \Omega_x(\bar{x}, \bar{y}) - \Omega_{xx}(\bar{x}, \bar{y}) \bar{x} - \Omega_{xy}(\bar{x}, \bar{y}) \bar{y}, \quad (44)$$

$$\tilde{g}_y(t) = \Omega_y(\bar{x}, \bar{y}) - \Omega_{xy}(\bar{x}, \bar{y}) \bar{x} - \Omega_{yy}(\bar{x}, \bar{y}) \bar{y}. \quad (45)$$

These are the “constants of linearization” — what you have to *add* to the linearized dynamics to reproduce the full nonlinear dynamics at the nominal state.

Remark. The rows are organised exactly as in Gino’s rendezvous MATLAB code `Rendezvous_Gino.m`: two position-dynamics blocks, two λ_r -dynamics blocks, two λ_v -dynamics blocks. The only differences are (i) a planar problem rather than 3D, so the block count drops from 9 to 6, and (ii) the HCW constant matrices M_{cw}, N_{cw} are replaced by the *time-varying* CR3BP Hessian entries evaluated on the nominal.

4.5 Number of unknowns and size of the linear system

For L ELM neurons per coordinate, n training points, and the planar problem:

unknowns $\boldsymbol{\xi}$	$6L$
equations	$6n$

We pick $n > L$ so the system is strictly overdetermined and well-posed for least-squares solution. Typical choices for a first Earth–Moon benchmark: $n = 40$, $L = 20$, giving a 240×120 linear system. The downstream Tikhonov+SVD step will then reduce this to a few dozen “effective” unknowns before binary encoding.

5 Linear LSQ $\rightarrow$ QUBO pipeline

This is the pipeline Gino introduced in `Rendezvous_GIno.m` and which the `qalunar.qubo.linear_lsq` module ports to Python. Given the linear system $\mathbf{A}\boldsymbol{\xi} = \mathbf{B}$ from Section 4.4, construct a quadratic unconstrained binary optimization (QUBO) whose ground state is the Tikhonov-regularized least-squares solution under a per-variable adaptive binary encoding.

5.1 Step 1 — Tikhonov regularization

Minimize

$$J(\boldsymbol{\xi}) = \|\mathbf{A}\boldsymbol{\xi} - \mathbf{B}\|^2 + \lambda \|\boldsymbol{\xi}\|^2, \quad (46)$$

which is the standard regularized least-squares objective. This is equivalent to the augmented system

$$\mathbf{A}_{\text{aug}} = \begin{pmatrix} \mathbf{A} \\ \sqrt{\lambda} \mathbf{I}_{6L} \end{pmatrix}, \quad \mathbf{B}_{\text{aug}} = \begin{pmatrix} \mathbf{B} \\ \mathbf{0} \end{pmatrix}, \quad (47)$$

and

$$\min_{\boldsymbol{\xi}} \|\mathbf{A}_{\text{aug}}\boldsymbol{\xi} - \mathbf{B}_{\text{aug}}\|^2. \quad (48)$$

The regularization strength is chosen relative to the spectrum of $\mathbf{A}$:

$$\lambda = \varepsilon_{\text{tik}} \cdot \sigma_{\max}(\mathbf{A})^2, \quad (49)$$

with $\varepsilon_{\text{tik}} = 10^{-6}$ in the default config. The scale $\sigma_{\max}^2$ makes the regularization self-calibrating to the problem size; ε_{tik} controls how much the regularizer bites into the solution.

Why bother with Tikhonov? Without it, rank-deficient or nearly-singular $\mathbf{A}$ (which is the rule rather than the exception when $n \rightarrow L$ or when the ELM spectrum decays fast) produces a solution with unboundedly large entries $\boldsymbol{\xi}$. Tikhonov forces the minimum-norm-*ish* solution and gives a *unique* $\boldsymbol{\xi}_{\text{tik}}$ that we can use as the encoding center.

5.2 Step 2 — Truncated SVD

Compute the thin SVD

$$\mathbf{A}_{\text{aug}} = \mathbf{U} \text{diag}(\mathbf{s}) \mathbf{V}^\top, \quad (50)$$

and keep only the singular values above a relative threshold:

$$r = \#\{i : s_i > \tau_{\text{svd}} s_1\}, \quad \tau_{\text{svd}} = 10^{-3} \text{ (default)}. \quad (51)$$

The right-basis columns $\mathbf{V}_r \in \mathbb{R}^{6L \times r}$ define a *reduced* parameterization

$$\boldsymbol{\xi} = \mathbf{V}_r \boldsymbol{\alpha}, \quad \boldsymbol{\alpha} \in \mathbb{R}^r, \quad (52)$$

with the Tikhonov solution in reduced coordinates

$$\boldsymbol{\alpha}_{\text{tik}} = \text{diag}(s_r)^{-1} \mathbf{U}_r^\top \mathbf{B}_{\text{aug}}, \quad (53)$$

where $\mathbf{U}_r$ and s_r are the first r columns and entries of $\mathbf{U}$ and $\mathbf{s}$. This typically cuts the number of variables from hundreds to tens: the binary encoding only has to cover the r -dimensional “active” subspace instead of the full $6L$ raw weights.

5.3 Step 3 — Per-variable adaptive encoding

Each reduced coordinate α_i is quantized over a personalized interval $[\alpha_{\text{tik},i} - R_i, \alpha_{\text{tik},i} + R_i]$ with p bits. The half-range R_i blends two contributions:

$$R_i = \text{margin} \cdot \left(|\alpha_{\text{tik},i}| + \frac{\|\mathbf{r}_{\text{aug}}\|}{s_i} \right), \quad \mathbf{r}_{\text{aug}} = \mathbf{B}_{\text{aug}} - \mathbf{A}_{\text{aug}} \boldsymbol{\xi}_{\text{tik}}, \quad (54)$$

and is then clipped below by `min_spread` to avoid degenerate zero-width intervals.

Why blend magnitude and sensitivity? The magnitude term $|\alpha_{\text{tik},i}|$ reserves enough room for the Tikhonov estimate itself (so the encoding isn't biased toward zero on large components). The sensitivity term $\|r\|/s_i$ reserves more room for small- s_i directions, which are the ones that can move a lot without changing the LSQ residual much; it is exactly the linear-sensitivity bound $|\delta\alpha_i| \leq \|\delta b\|/s_i$ for a perturbed right-hand side.

The encoding is then

$$\alpha_i(\mathbf{q}) = \alpha_{\text{offset},i} + \text{step}_i \sum_{k=0}^{p-1} 2^k q_{ip+k}, \quad \text{step}_i = \frac{2R_i}{2^p - 1}, \quad \alpha_{\text{offset},i} = \alpha_{\text{tik},i} - R_i. \quad (55)$$

In matrix form, $\boldsymbol{\alpha} = \boldsymbol{\alpha}_{\text{offset}} + \mathbf{D}_{\text{dec}} \mathbf{q}$, with $\mathbf{D}_{\text{dec}} \in \mathbb{R}^{r \times rp}$ block-diagonal with blocks $\text{step}_i \cdot (1, 2, 4, \dots, 2^{p-1})$.

5.4 Step 4 — QUBO assembly

Substituting the encoding into the Tikhonov objective:

$$J(\mathbf{q}) = \|\mathbf{A}_{\text{aug}} \mathbf{V}_r (\boldsymbol{\alpha}_{\text{offset}} + \mathbf{D}_{\text{dec}} \mathbf{q}) - \mathbf{B}_{\text{aug}}\|^2 \quad (56)$$

$$= \|\Phi \mathbf{q} - \tilde{\mathbf{B}}\|^2, \quad (57)$$

where

$$\mathbf{A}_{\text{red}} = \mathbf{A}_{\text{aug}} \mathbf{V}_r, \quad (58)$$

$$\Phi = \mathbf{A}_{\text{red}} \mathbf{D}_{\text{dec}}, \quad (59)$$

$$\tilde{\mathbf{B}} = \mathbf{B}_{\text{aug}} - \mathbf{A}_{\text{red}} \boldsymbol{\alpha}_{\text{offset}}. \quad (60)$$

Expanding the square gives the QUBO

$$E(\mathbf{q}) = \mathbf{q}^\top \mathbf{Q} \mathbf{q} + \boldsymbol{\ell}^\top \mathbf{q} + c, \quad (61)$$

with

$$\mathbf{Q} = \Phi^\top \Phi, \quad \boldsymbol{\ell} = -2 \Phi^\top \tilde{\mathbf{B}}, \quad c = \|\tilde{\mathbf{B}}\|^2. \quad (62)$$

The core identity. By construction,

$$E(\mathbf{q}) = \|\mathbf{A}_{\text{aug}} \boldsymbol{\xi}(\mathbf{q}) - \mathbf{B}_{\text{aug}}\|^2 \quad \text{for every } \mathbf{q} \in \{0, 1\}^{rp}. \quad (63)$$

That is, the QUBO ground state *is* the best binary-encodable approximation to the Tikhonov-regularized LSQ solution — no approximation error beyond quantization. The unit tests in `tests/test_qubo_linear_lsq.py` verify this identity on random bit vectors.

5.5 Decoding and warm starts

To go from a bitstring $\mathbf{q}$ back to the original unknowns:

$$\mathbf{q} \longrightarrow \boldsymbol{\alpha} = \boldsymbol{\alpha}_{\text{offset}} + \mathbf{D}_{\text{dec}} \mathbf{q} \longrightarrow \boldsymbol{\xi} = \mathbf{V}_r \boldsymbol{\alpha}. \quad (64)$$

The **warm start** is the quantization of $\boldsymbol{\alpha}_{\text{tik}}$ to p bits per coordinate. Because the encoding is centered at $\boldsymbol{\alpha}_{\text{tik}}$, the warm start is within half a quantization step per coordinate of the Tikhonov solution, and the corresponding bitstring is a strong initial seed for D-Wave hybrid solvers or simulated annealing.

5.6 Default hyperparameters

Parameter	Symbol	Default
Bits per reduced variable	p	8
Tikhonov (relative to σ_{max}^2)	ε_{tik}	10^{-6}
SVD truncation threshold	τ_{svd}	10^{-3}
Encoding margin	—	3.0
Minimum half-range floor	—	10^{-6}

These match Gino’s MATLAB defaults and are the starting point for the sensitivity sweep once the Earth–Moon benchmark runs.

6 Sequential linearization outer loop

Because the CR3BP is nonlinear, one call to the QUBO inner solver does not give the optimal trajectory, it gives the optimal *linear perturbation* around whatever nominal was used to build $\mathbf{A}, \mathbf{B}$. The sequential outer loop iterates:

1. Start with an initial nominal $\bar{\mathbf{x}}^{(0)}(t)$ (e.g. the Hermite cubic interpolant of the boundary conditions, or a classical-NLP reference solution).
2. At iteration k :
 - (a) Evaluate $\Omega_{xx}, \Omega_{xy}, \Omega_{yy}$ and the forcing terms $\tilde{g}_x, \tilde{g}_y$ on the nominal.
 - (b) Assemble the linear system $\mathbf{A}^{(k)}\boldsymbol{\xi} = \mathbf{B}^{(k)}$.
 - (c) Build and solve the QUBO; decode to $\boldsymbol{\xi}^{(k)}$ and reconstruct the new trajectory $\mathbf{x}^{(k+1)}(t)$.
 - (d) Update the nominal: $\bar{\mathbf{x}}^{(k+1)} = \mathbf{x}^{(k+1)}$.
3. Repeat until $\|\bar{\mathbf{x}}^{(k+1)} - \bar{\mathbf{x}}^{(k)}\|$ drops below a tolerance, or the cost J stops decreasing.

This is essentially a Gauss–Newton / sequential linearization approach, very similar to what sequential convex programming (SCP) does in the classical trajectory-optimization literature. The difference is that the inner step is not a convex QP solve but a QUBO ground-state search.

Pass-1 vs. Pass-2 loop. The implementation in `IndirectTfcElmTranscription.solve_sequential` supports two modes, selected by the flag `OuterLoopConfig.line_search`:

- **Pass 1** (`line_search=False`): fixed damping, $\bar{\mathbf{x}}^{(k+1)} = \bar{\mathbf{x}}^{(k)} + \alpha(\mathbf{x}^{(k+1)} - \bar{\mathbf{x}}^{(k)})$ with a constant $\alpha \in (0, 1]$. Reproduces the behaviour of the first-pass rendezvous loop. Tolerant of mild nonlinearity; oscillates or diverges on stronger CR3BP transfers.
- **Pass 2** (`line_search=True`, the default): adaptive Armijo backtracking with merit function $m(\bar{\mathbf{x}}) = \|\mathbf{x}(\boldsymbol{\xi}(\bar{\mathbf{x}})) - \bar{\mathbf{x}}\|_\infty$, i.e. the *fixed-point residual* of the Picard map $\bar{\mathbf{x}} \mapsto \mathbf{x}(\boldsymbol{\xi}(\bar{\mathbf{x}}))$, which is the right Newton merit for the rootfinding problem $F(\bar{\mathbf{x}}) := \mathbf{x}(\boldsymbol{\xi}(\bar{\mathbf{x}})) - \bar{\mathbf{x}} = \mathbf{0}$.

6.1 Pass-2 Armijo line search

The full step at iteration k is

$$\Delta\bar{\mathbf{x}}^{(k)} = \mathbf{x}^{(k+1)} - \bar{\mathbf{x}}^{(k)}.$$

Starting from $\alpha = \text{damping}$ (default 1.0 — a full Newton-like step), we trial

$$\bar{\mathbf{x}}_{\text{trial}}^{(k+1)} = \bar{\mathbf{x}}^{(k)} + \alpha \Delta\bar{\mathbf{x}}^{(k)}$$

and accept it when

$$m(\bar{\mathbf{x}}_{\text{trial}}^{(k+1)}) \leq (1 - c_1 \alpha) m(\bar{\mathbf{x}}^{(k)}), \tag{65}$$

with $c_1 = \text{armijo_c} = 10^{-4}$. Otherwise α is multiplied by `backtrack_factor` (default 1/2) and the trial is repeated, down to a floor $\alpha \geq \text{min_damping}$ (default 2^{-10}).

Why the fixed-point residual? The nonlinear TPBVP residual is *not* a valid Armijo merit here: the linearization changes at every trial nominal, so $\|R_{\text{NL}}(\boldsymbol{\xi}^{\text{trial}})\|$ need not be monotone with α . The fixed-point residual $\|F(\bar{\mathbf{x}})\|_\infty$, on the other hand, is exactly the quantity the outer loop is trying to zero; it is the analogue of $\|\nabla f\|$ in gradient descent, and its Armijo-like decrease gives the outer loop a globalization guarantee — once Pass 2 has accepted a step, the fixed-point residual cannot grow above its initial value.

Concrete effect. On the short, gentle fixture used in `tests/test_indirect_tfc_elm.py`, Pass 2 accepts $\alpha = 1$ at every iteration and converges in about 13 outer iterations, versus ~ 32 for fixed damping $\alpha = 0.5$. On the long-transfer fixture, fixed-damping $\alpha = 1$ oscillates wildly — peak step norm ≈ 2 against an initial value ≈ 0.7 — while the Armijo-bounded version keeps the step norm under its initial value for all 40 iterations, as the test `TestLineSearch::test_line_search_bounds_step_norm_on_aggressive_problem` verifies. That said, bounding the step norm is not the same as converging: on the long transfer, even the line-searched loop stalls outside the basin of attraction if seeded only by the BC-only Hermite interpolant, which motivates the classical reference of the next section.

7 Classical reference via direct collocation

The outer loop of Section 6 is a contraction *only inside a local basin of attraction*, and the basin can be genuinely small on long or curved transfers in the Earth–Moon CR3BP. The indirect transcription needs a good initial nominal, and the BC-only Hermite interpolant `initial_nominal()` is not good enough once the trajectory makes any real excursion away from the straight line joining the endpoints.

We therefore ship a **classical direct-collocation reference solver** in `qalunar.reference.direct_collocation`. It plays two distinct roles in the project:

1. **Warm start.** The solution produced by the reference solver is resampled onto the TFC+ELM training grid and passed as `initial_nominal` to `solve_sequential`.
2. **Classical baseline.** The same solver provides a CPU reference objective J_{NLP}^* that the QUBO samples will be measured against in the benchmark sweep (project Task 9).

7.1 Hermite–Simpson direct collocation

Split $[0, T]$ into N uniform intervals of width $h = T/N$ and introduce node values at $N + 1$ points,

$$\mathbf{x}_k = (x_k, y_k, v_{x,k}, v_{y,k}), \quad \mathbf{u}_k = (u_{x,k}, u_{y,k}), \quad k = 0, \dots, N.$$

The decision vector is the concatenation

$$\mathbf{z} = (\mathbf{x}_0, \dots, \mathbf{x}_N, \mathbf{u}_0, \dots, \mathbf{u}_N) \in \mathbb{R}^{6(N+1)}.$$

Hermite–Simpson defect. On each sub-interval $[t_k, t_{k+1}]$, the cubic Hermite interpolant through $(\mathbf{x}_k, \mathbf{f}_k)$ and $(\mathbf{x}_{k+1}, \mathbf{f}_{k+1})$ must agree with the dynamics at the midpoint. With $\mathbf{f}_k = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$,

$$\mathbf{x}_{k+1/2} = \frac{1}{2}(\mathbf{x}_k + \mathbf{x}_{k+1}) + \frac{h}{8}(\mathbf{f}_k - \mathbf{f}_{k+1}), \quad (66)$$

$$\mathbf{u}_{k+1/2} = \frac{1}{2}(\mathbf{u}_k + \mathbf{u}_{k+1}), \quad (67)$$

$$\mathbf{f}_{k+1/2} = \mathbf{f}(\mathbf{x}_{k+1/2}, \mathbf{u}_{k+1/2}). \quad (68)$$

The Hermite–Simpson defect constraint is

$$\boxed{\mathbf{d}_k = \mathbf{x}_{k+1} - \mathbf{x}_k - \frac{h}{6}(\mathbf{f}_k + 4\mathbf{f}_{k+1/2} + \mathbf{f}_{k+1}) = \mathbf{0}}, \quad k = 0, \dots, N-1. \quad (69)$$

This is a third-order collocation scheme, and it is a standard choice from Kelly [6] and Betts [7].

Objective. The energy cost is discretized with the composite trapezoidal rule,

$$\tilde{J}(\mathbf{z}) = \frac{h}{2} \sum_{k=0}^N w_k \|\mathbf{u}_k\|^2, \quad w_0 = w_N = \frac{1}{2}, \quad w_k = 1 \text{ otherwise.} \quad (70)$$

$\tilde{J}$ is a pure quadratic in the control block (Hessian constant and sparse), and its gradient

$$\partial \tilde{J} / \partial \mathbf{u}_k = h w_k \mathbf{u}_k$$

is handed analytically to SLSQP; only the defect Jacobian is finite-differenced.

Boundary conditions and bounds. The eight equalities $\mathbf{x}_0 = (\mathbf{r}_0, \mathbf{v}_0)$ and $\mathbf{x}_N = (\mathbf{r}_f, \mathbf{v}_f)$ are appended to the constraint list. An optional symmetric box $|u_{x,k}|, |u_{y,k}| \leq u_{\max}$ can be passed via `DirectCollocationConfig.control_bound`; states are left unbounded.

The full NLP is

$$\min_{\mathbf{z}} \tilde{J}(\mathbf{z}) \quad \text{s.t.} \quad \mathbf{d}_k(\mathbf{z}) = \mathbf{0} \quad \forall k, \quad \mathbf{x}_0 = \mathbf{s}_0, \quad \mathbf{x}_N = \mathbf{s}_f, \quad \mathbf{u}_k \in [-u_{\max}, u_{\max}]^2, \quad (71)$$

and is handed to `scipy.optimize.minimize` with `method="SLSQP"`. With $N = 40$ the problem has a few hundred variables and constraints and solves in well under a second.

7.2 Resampling onto the TFC+ELM grid

The direct-NLP result is returned as `DirectCollocationResult`, which exposes a helper

```
sample_position(t_query) -> (x_bar, y_bar)
```

that linearly interpolates the node positions onto an arbitrary time grid. Warm-starting the indirect solver is then a three-liner:

```
1 ref = solve_energy_optimal_cr3bp(dynamics, r0, v0, rf, vf, T)
2 x_bar, y_bar = ref.sample_position(transcription.t)
3 sol = transcription.solve_sequential(
4     OuterLoopConfig(), initial_nominal=(x_bar, y_bar),
5 )
```

Linear interpolation is enough because the outer loop uses the nominal only to evaluate $\Omega_{xx}, \Omega_{xy}, \Omega_{yy}$ and $\tilde{g}_x, \tilde{g}_y$ pointwise; those are smooth functions of (x, y) and insensitive to sub-node interpolation error.

7.3 Example trajectories: a two-phase Earth–Moon mission

A physically realistic LEO-to-lunar low-thrust transfer is a multi-revolution spiral with hundreds of orbits around Earth over months to years of flight, well beyond what a single polynomial (or ELM) expansion can represent. Rather than ask the CR3BP transcription to do work it cannot do, we split the mission in two:

- **Phase 1** — analytical Edelbaum spiral from a low circular Earth orbit out to a high circular Earth orbit, ignoring lunar gravity. In this regime the dynamics are pure two-body Keplerian around Earth and the problem is closed-form: tangential thrust is already Edelbaum-optimal for a coplanar circular-to-circular transfer, so there is nothing to optimize. We read off Δv , time-of-flight and revolution count from the classical formulas and move on.
- **Phase 2** — CR3BP energy-optimal transfer from the Phase-1 handoff state to a rest rendezvous west of the Moon, solved by the Hermite–Simpson direct NLP described above (and, later, by the TFC+ELM indirect transcription of Section 4). This is the three-body leg that the whole QUBO stack targets.

Phase 1 is implemented in `qalunar.reference.edelbaum` as a ten-line closed-form function `edelbaum_spiral( $r_1, r_2, a_t$ )` that returns an `EdelbaumSpiral` dataclass. In CR3BP nondimensional units, with $GM_\oplus = 1 - \mu$, the inertial circular speed is $v_i = \sqrt{(1 - \mu)/r_i}$, so the total Δv is just $|v_1 - v_2|$, the time of flight is $|v_1 - v_2|/a_t$ (constant tangential thrust integrates trivially), and the revolution count comes from integrating the orbital angular rate $v/r = v^3/GM$ against dt :

$$N_{\text{rev}} = \frac{|v_1^4 - v_2^4|}{8\pi(1 - \mu)a_t}. \quad (72)$$

The `handoff_state_synodic()` method then places the spacecraft on the $+x$ direction at the moment its Earth-relative position crosses the positive x -axis, so its synodic position is $(-\mu + r_2, 0)$ with inertial velocity $(0, v_2)$ prograde, mapped to synodic velocity by subtracting $\boldsymbol{\omega} \times \mathbf{r} = (-y, x)$.

The reference mission. The demo hero mission is a spiral from LEO ($r_1 = 0.018$ nondim, ~ 6900 km) to a high circular Earth orbit at $r_2 = 0.5$ nondim (~ 192000 km, half the Earth–Moon distance), at a plausible solar-electric thrust acceleration of 0.1 mm/s^2 . Phase 1 yields (all numbers from `EdelbaumSpiral.si_report()`):

- $\Delta v_{\text{P1}} \approx 6142 \text{ m/s}$,
- $T_{\text{P1}} \approx 709 \text{ days} \approx 1.94 \text{ yr}$,
- $N_{\text{rev}} \approx 3292$ revolutions around Earth.

The handoff state is $\mathbf{r}_0 = (-\mu + 0.5, 0) \approx (0.488, 0)$ with synodic velocity $\mathbf{v}_0 = (0, v_2 - (-\mu + r_2)) \approx (0, 0.918)$ — a prograde tangential crossing of the $+x$ axis on the Phase-1 exit orbit. Phase 2 then runs the direct NLP from this state to a rest rendezvous at $\mathbf{r}_f = (1 - \mu - 0.1, 0)$, $\mathbf{v}_f = \mathbf{0}$, with flight time $T = 3.0$ nondim (≈ 13 days in physical units). SLSQP drives the Hermite–Simpson defects to $\approx 3 \cdot 10^{-10}$

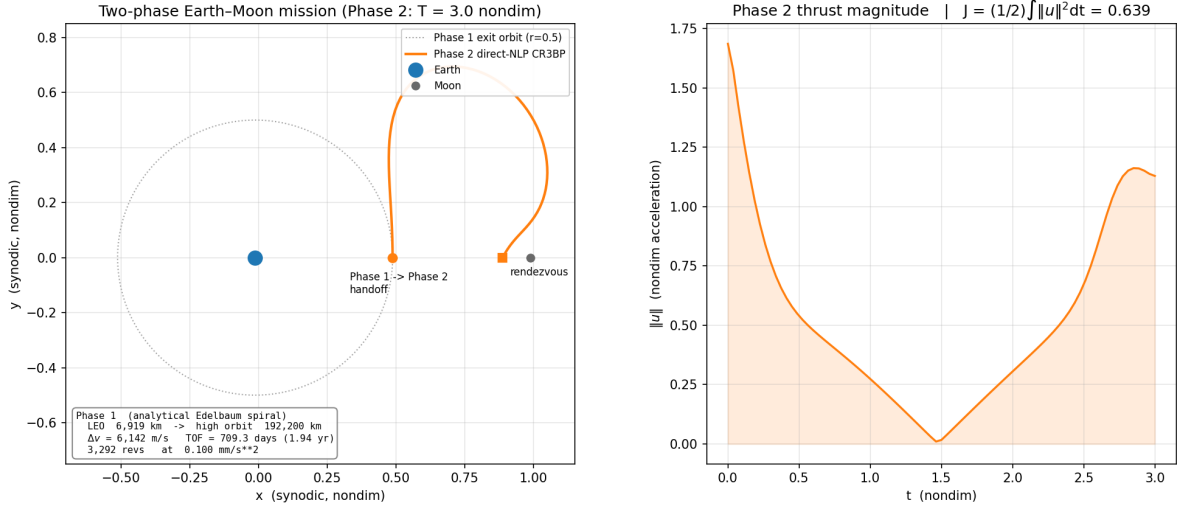


Figure 1: Two-phase Earth–Moon low-thrust mission. Phase 1 (dashed gray) is an analytical Edelbaum spiral from LEO out to a $r = 0.5$ nondim circular Earth orbit, summarized in the caption box; the spiral itself is hundreds of revolutions and is not drawn explicitly. Phase 2 (orange) is the direct-NLP energy-optimal CR3BP transfer from the Phase-1 handoff state $(0.488, 0)$ with synodic velocity $(0, 0.918)$ to a rest rendezvous at $(1 - \mu - 0.1, 0)$, flight time $T = 3.0$ nondim. Left: position in the synodic frame, with Earth and Moon markers. Right: Phase-2 thrust-acceleration magnitude, showing a smooth push-brake profile with peak ≈ 1.69 nondim and objective $J \approx 0.64$. The Phase-1 summary box reports $\Delta v = 6\,142$ m/s, $T = 709$ days, $N_{\text{rev}} = 3\,292$ at 0.1 mm/s².

in 53 iterations with objective $J = \frac{1}{2} \int_0^T ||u||^2 dt \approx 0.64$ and peak acceleration $||u||_{\text{max}} \approx 1.69$ nondim. The resulting arc is a single prograde sweep from the Phase-1 exit orbit to the Moon realm, shown in Figure 1.

A second, deliberately synthetic benchmark (the same BCs used by the `transcription` test fixture: $\mathbf{r}_0 = (-0.3, 0)$, $\mathbf{v}_0 = (0, 0.6)$, $\mathbf{r}_f = (0.4, 0.2)$, $\mathbf{v}_f = (-0.1, 0)$, $T = 2.5$) is what exhibits the warm-start advantage. These BCs were chosen to keep the trajectory *away* from both primaries so that the Hessian entries $\Omega_{xx}, \Omega_{xy}, \Omega_{yy}$ (which scale as $\sim 1/r^3$ near each primary) stay bounded and unit-test conditioning is predictable — they are a numerical stress test for the indirect iteration, not a mission geometry. On the two-phase Earth–Moon mission of Figure 1, direct NLP and the indirect TPBVP sit in nearby basins and cold-start already contracts almost as well as warm-start, so the warm-start *advantage* is best displayed on the synthetic benchmark, where cold-start is catastrophically stuck. The synthetic-benchmark comparison overlays, in the synodic frame:

- the direct-NLP reference (orange),
- the cold-started indirect iteration — BC-only Hermite seed, no warm start — after 40 outer iterations (red dashed),
- the indirect iteration warm-started from `ref.sample_position(transcription.t)` (green).

The cold-started indirect trajectory veers off into a completely unphysical arc: the sequential linearization is stuck outside the basin of attraction and never recovers, exactly the failure mode the Pass-2 Armijo line search *bounds* but does not cure. The warm-started iteration, in contrast, tracks the classical reference within a handful of outer iterations.

Quantitatively, the nonlinear TPBVP residual history for the two seeds tracks the same pattern. The BC-only seed plateaus at $||R_{\text{NL}}||_{\infty} \approx 13$ after two iterations and never moves. The direct-NLP warm start drops from ≈ 30 to ≈ 0.74 in three iterations — roughly a $17\times$ reduction — and plateaus there. The remaining ~ 0.74 residual is the last obstacle: it is a fixed point of the *linearization* whose Ω -Hessian freezing drift is small enough to be in the basin of the true TPBVP but too large to be zeroed by the pure Picard structure of the outer loop, which only relinearizes at committed nominals and therefore carries stale A, B matrices inside each Armijo trial step. Cracking this remainder motivates a Levenberg–Marquardt / Gauss–Newton extension of `solve_sequential` that relinearizes at every trial step during the backtrack; a direct-LGL pseudospectral transcription is held as a backup baseline for comparison. Project Task 8 will report whether the ~ 0.74 remainder matters for QUBO quality in practice.

8 Code organization

The codebase lives under `qalunar/` and is organized by separation of concerns:

```
1 qalunar/
2   __init__.py
3   dynamics/
4     __init__.py
5     cr3bp.py           # PlanarCR3BP: rhs, Jacobian, Jacobi integral
6   qubo/
7     __init__.py
8     linear_lsq.py      # build_linear_lsq_qubo + LinearLsqToQuboResult
9   transcription/
10    __init__.py
11    indirect_tfc_elm.py # IndirectTfcElmTranscription + OuterLoopConfig
12  reference/
13    __init__.py
14    direct_collocation.py # Hermite-Simpson NLP (warm start + baseline)
15    edelbaum.py         # analytical Phase-1 Earth-escape spiral
16 tests/
17   test_cr3bp.py        # 18 tests: L4/L5, df/dx vs FD, ...
18   test_qubo_linear_lsq.py # 24 tests: energy identity, warm start
19   test_indirect_tfc_elm.py # 51 tests: TFC, ELM, outer loop, line
20     search
21   test_reference_direct_collocation.py # 20 tests: HS defects, warm-start
22   test_reference_edelbaum.py          # 26 tests: closed-form spiral, handoff
23 scripts/
24   plot_cr3bp_overview.py              # primaries, L4/L5, ZVCs
25   plot_reference_trajectories.py      # Section 7 figures
26 docs/
27   qalunar_manual.tex                 # this document
```

8.1 Running the tests

```
1 # from the project root
2 python -m pytest -q
```

The full suite currently contains 139 tests and runs in roughly 2 minutes (the bulk of the runtime is the SLSQP solves inside the direct-collocation tests). It is the first place to look when something is misbehaving: every mathematical identity in this manual corresponds to at least one executable assertion.

8.2 Key modules at a glance

qalunar.dynamics.cr3bp. Planar CR3BP with a 2D thrust control. `PlanarCR3BP.rhs` and `jacobian_state` implement equations (5) and the state Jacobian used for linearization. A `lagrange_triangular_points` helper returns L_4 , L_5 symbolically. The Earth–Moon mass parameter is exposed as the module constant `EARTH_MOON_MU`.

qalunar.qubo.linear_lsq. One-function entry point: `build_linear_lsq_qubo(A, B, config)`. Returns a `LinearLsqToQuboResult` bundling the QUBO (Q, ℓ, c) , the decoding path $(V_r, \alpha_{\text{offset}}, \text{step}_i)$, and diagnostics (singular spectrum, effective rank, Tikhonov solution, λ_{tik}). Methods: `decode_alpha`, `decode_xi`, `energy`, `warm_start_bits`.

qalunar.transcription.indirect_tfc_elm. The indirect TFC+ELM pipeline of Sections 4–6. `IndirectTfcElmTrans` assembles the linear system of (43) for a given nominal via `build_linear_system`, and `solve_sequential(cfg, initial_nominal=...)` runs the outer loop: classical LSQ solve, decode, Pass-2 Armijo backtracking (Section 6.1), update nominal. Returns a `SequentialSolution` with `xi`, `trajectory` and a per-iteration history dict containing `trajectory_change_inf`, `linearized_residual`, `nonlinear_residual` and `step_size`.

qalunar.reference.direct_collocation. The classical Hermite–Simpson direct collocation solver of Section 7. Entry points: `DirectCollocationConfig`, `DirectCollocationResult` and the top-level function `solve_energy_optimal_cr3bp(dynamics, r0, v0, rf, vf, T, config)`. The returned result carries `t`, `x`, `y`, `vx`, `vy`, `ux`, `uy`, the SLSQP convergence status, and a `sample_position(t_query)` helper used to warm-start `IndirectTfcElmTranscription.solve_sequential`.

qalunar.reference.edelbaum. Closed-form analytical solver for the Phase-1 Earth-escape spiral (Section 7.3). `edelbaum_spiral(r1, r2, a_thrust, mu)` returns an `EdelbaumSpiral` dataclass holding $r_i, v_i, \Delta v, T, N_{\text{rev}}, a_t$ in CR3BP nondimensional units. The `handoff_state_synodic()` method yields the synodic $(\mathbf{r}, \mathbf{v})$ initial state for the Phase-2 direct NLP, and `si_report()` returns a dict of pre-formatted SI-unit strings (Δv in m/s, TOF in days, etc.) for manual captions and progress logs.

The indirect TFC+ELM pipeline produces a linear system $\mathbf{A}\boldsymbol{\xi} = \mathbf{B}$ with decoding metadata compatible with `qubo.linear_lsqr`; a direct-LGL pipeline will land alongside it in `qalunar.transcription` as part of project Task 8.

9 Convergence investigation and pivot

The sequential linearization loop of the hybrid QUBO–classical pipeline was expected to converge toward the known direct-NLP solution. Extensive numerical experiments revealed two orthogonal failure modes that together make the continuous-lstsqr QUBO approach unsuitable for production-quality trajectory optimization.

9.1 Armijo line search failure

The Armijo backtracking strategy (Pass 2) uses the fixed-point residual $m(\bar{\mathbf{x}}) = \|\mathbf{x}(\boldsymbol{\xi}(\bar{\mathbf{x}})) - \bar{\mathbf{x}}\|_\infty$ as merit function. Diagnostic runs showed that this merit function is too noisy for the Picard map in the CR3BP: the Armijo condition is never satisfied at large step sizes, forcing α down to the floor $\alpha_{\min} = 0.001$ at every iteration. The effective update becomes negligibly small, and the nonlinear residual stagnates rather than converging.

In contrast, *undamped Picard iteration* (fixed $\alpha = 1$, no line search) converges reliably on the same problems. Using the classical NLP solution as an initial nominal and solving the inner lstsq problem exactly (not via QUBO), undamped Picard reduces the nonlinear residual from 5.7 to 1.9×10^{-5} in fewer than 10 iterations and 0.2 seconds of wall time.

Remark. This confirms that the sequential linearization itself is sound; the inner linear solve and the outer Picard update converge. The Armijo mechanism was the bottleneck, not the linearization quality.

9.2 QUBO quantization error

Even after removing the Armijo failure (using $\alpha = 1$), replacing the exact lstsq inner solve with the QUBO pipeline introduces a discretization error that prevents convergence.

Each continuous reduced variable α_j is encoded as a p -bit binary expansion (Section 5):

$$\alpha_j \approx \alpha_j^{\min} + \Delta_j \sum_{k=0}^{p-1} 2^k q_{jp+k}, \quad \Delta_j = \frac{\alpha_j^{\max} - \alpha_j^{\min}}{2^p - 1}.$$

With $p = 8$ bits, each variable has a resolution of roughly $\Delta_j/256$. The QUBO ground state finds the best *representable* α , which generically differs from the exact lstsq minimizer by $O(\Delta_j)$.

At Picard iteration k , the update reconstructed from the QUBO solution carries this quantization error forward into the next nominal. Because the linearization point has shifted by a quantization-scale perturbation rather than the true Newton step, subsequent iterations cannot reduce the residual below a floor set by Δ_j . Numerically, the hybrid loop with 8-bit encoding oscillates between nonlinear residuals of ~ 7 –12 indefinitely, never approaching the 10^{-5} achievable by exact lstsq.

Increasing p would shrink Δ_j but expands the QUBO dimension (number of qubits) proportionally. At $p = 16$, the QUBO for a problem with $r = 30$ reduced variables already requires 480 qubits and a 480×480 coupling matrix, and even 16-bit resolution ($\sim 10^{-5}$ per variable) may not suffice for demanding transfers.

9.3 Why polishing does not save the approach

One could “polish” the QUBO output by running a classical lstsq step on the decoded continuous solution. But this is scientifically misleading: the lstsq polish does the actual work of finding the continuous minimum, while the QUBO merely provides a warm start that the lstsq solver does not need. Classical lstsq solves the inner problem in ~ 5 ms; the QUBO *approximates* the same answer in orders of magnitude more time.

9.4 Conclusion: the continuous-lstsq QUBO is the wrong problem

The Carbone et al. approach [1] works for the particular problem it was designed for (HCW rendezvous with a single linear solve), but does not generalize well to nonlinear problems requiring iterative refinement. The fundamental mismatch is that QUBO variables are binary, yet the underlying optimization variables are continuous. Artificially encoding continuous variables as bitstrings introduces quantization error that dominates the solution quality.

This motivates a search for problems where the decision variables are *inherently binary*, so that the QUBO formulation is exact rather than an approximation.

10 On/off thrust scheduling via QUBO

Low-thrust engines can operate in two modes: on (full thrust) or off (coasting). Given a transfer window divided into N time steps, the decision “fire at step i ?” is a naturally binary variable $q_i \in \{0, 1\}$. The search space of 2^N possible schedules is combinatorially hard for classical exhaustive search, making this an ideal candidate for quantum annealing.

10.1 Problem setup

Consider a spacecraft in the planar CR3BP with initial state $\mathbf{x}_0 = (x_0, y_0, \dot{x}_0, \dot{y}_0)^\top$. The time interval $[t_0, t_f]$ is divided into N decision intervals of length $\Delta t = (t_f - t_0)/N$. At each interval i , the engine is either on ($q_i = 1$, applying thrust acceleration $\mathbf{u}_i$) or off ($q_i = 0$, coasting).

The thrust direction at each step can be:

- **tangential** — along the velocity vector,
- **anti-tangential** — opposite to the velocity vector, or
- **fixed** — a constant direction $\hat{\mathbf{u}}$ specified by the user.

In all cases, the magnitude $\|\mathbf{u}_i\| = a_T$ is constant (on/off model: full thrust or zero).

10.2 Linearization via the state transition matrix

To formulate a QUBO, the effect of each burn on the final state must be expressed as a linear function of the binary variables q_i .

Coast arc and STM. First, propagate the uncontrolled (coast) trajectory from $\mathbf{x}_0$ over $[t_0, t_f]$ using the CR3BP equations of motion. Simultaneously integrate the state transition matrix (STM) via the variational equations:

$$\dot{\Phi}(t, t_0) = \mathbf{A}(t) \Phi(t, t_0), \quad \Phi(t_0, t_0) = \mathbf{I}_{4 \times 4},$$

where $\mathbf{A}(t)$ is the Jacobian of the CR3BP right-hand side evaluated along the coast arc. The implementation integrates the augmented 20-dimensional system $\mathbf{z} = (\mathbf{x}, \text{vec}(\Phi))$ using fixed-step RK4.

The STM $\Phi(t_i, t_f)$ from decision point t_i to the final time is obtained by composition:

$$\Phi(t_i, t_f) = \Phi(t_f, t_0) \Phi(t_i, t_0)^{-1}.$$

Impulse response vectors. The control influence matrix for the planar CR3BP is

$$\mathbf{B}_{\text{ctrl}} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix},$$

so an impulsive thrust acceleration $\mathbf{u}_i$ at time t_i produces a first-order change at t_f of:

$$\mathbf{b}_i = \Phi(t_i, t_f) \mathbf{B}_{\text{ctrl}} \mathbf{u}_i \Delta t.$$

The vector $\mathbf{b}_i \in \mathbb{R}^4$ encodes the full effect of burning at step i on the final position and velocity.

Total effect of a schedule. The superposition principle (valid under the linearization) gives the final-state deviation from the coast arc as:

$$\delta \mathbf{x}(t_f) \approx \sum_{i=1}^N q_i \mathbf{b}_i.$$

10.3 QUBO formulation

Let $\mathbf{d} = \mathbf{x}_{\text{target}} - \mathbf{x}_{\text{coast}}(t_f)$ be the state gap that the burns must close. The objective balances trajectory accuracy against fuel usage:

$$f(\mathbf{q}) = \underbrace{\left\| \sum_{i=1}^N q_i \mathbf{b}_i - \mathbf{d} \right\|_{\mathbf{W}}^2}_{\text{miss distance}} + \underbrace{\alpha \sum_{i=1}^N q_i}_{\text{fuel penalty}}, \quad (73)$$

where $\mathbf{W} = \text{diag}(w_1, w_2, w_3, w_4)$ allows per-component weighting (e.g. position only) and $\alpha \geq 0$ penalizes the number of burn segments.

Expanding the quadratic norm and using $q_i^2 = q_i$ for binary variables:

$$Q_{ij} = \mathbf{b}_i^\top \mathbf{W} \mathbf{b}_j, \quad \text{quadratic coupling (symmetric),} \quad (74)$$

$$\ell_i = \alpha - 2 \mathbf{b}_i^\top \mathbf{W} \mathbf{d}, \quad \text{linear term,} \quad (75)$$

$$c = \mathbf{d}^\top \mathbf{W} \mathbf{d}, \quad \text{constant offset.} \quad (76)$$

The QUBO energy is then $f(\mathbf{q}) = \mathbf{q}^\top \mathbf{Q} \mathbf{q} + \boldsymbol{\ell}^\top \mathbf{q} + c$.

Remark. Unlike the continuous-lstsq QUBO of Section 5, there is no quantization error whatsoever: every q_i is a physical binary decision, and the QUBO exactly encodes the scheduling objective (73). The only approximation is the linearization of thrust effects via the STM, which is standard practice in preliminary mission design.

10.4 Solvers

Brute force. For $N \leq 20$ ($2^{20} \approx 10^6$ evaluations), the global optimum is found by exhaustive enumeration. This provides a ground truth against which heuristic solvers are validated.

Simulated annealing (SA). The QUBO is converted to a `dimod.BinaryQuadraticModel` and sampled using `neal.SimulatedAnnealingSampler`. SA matches the brute-force optimum on all problem sizes tested ($N = 8$ to 20) with 500–1000 reads.

D-Wave quantum processing unit (QPU). The scheduling QUBO maps directly to D-Wave hardware: each decision step is one physical qubit, and the Q_{ij} couplings are programmed onto the Pegasus/Zephyr topology. With N qubits, the problem fits comfortably within the Advantage2 processor’s 5000+ qubits, even for N in the hundreds.

Mixed-integer linear programming (MILP) baseline. A classical, provably optimal baseline is obtained by lifting the binary-quadratic objective to a pure MILP via the McCormick linearization $y_{ij} \leq q_i$, $y_{ij} \leq q_j$, $y_{ij} \geq q_i + q_j - 1$, $y_{ij} \in \{0, 1\}$, which exactly enforces $y_{ij} = q_i q_j$. Substituting $q_i^2 = q_i$ for binary variables, the QUBO energy becomes

$$E(\mathbf{q}) = \sum_i (Q_{ii} + l_i) q_i + \sum_{i < j} 2 Q_{ij} y_{ij} + \text{const},$$

which is linear in $(\mathbf{q}, \mathbf{y})$ and solvable by HiGHS through `scipy.optimize.milp`. This provides certified optimality for small to medium N and a neutral classical scaling reference.

10.5 Iterative re-linearization

The QUBO uses a single first-order STM expansion around a reference trajectory. For a long arc or a strong thrust the linearization error accumulates: the schedule that minimizes the linearized objective need not minimize the true nonlinear miss, and may even *increase* it. We address this with a sequential re-linearization driver, the binary analogue of the continuous-side Pass-2 loop in Section 6.

Let $\mathbf{q}^{(k)}$ be the schedule at iteration k and let $\mathbf{x}_{\text{nom}}^{(k)}(t)$ be the trajectory obtained by propagating the true nonlinear CR3BP under the controls implied by $\mathbf{q}^{(k)}$. We linearize the final state as

$$\mathbf{x}(t_f) \approx \mathbf{x}_{\text{nom}}^{(k)}(t_f) + \sum_j (q_j - q_j^{(k)}) \mathbf{b}_j^{(k)}, \quad (77)$$

where $\mathbf{b}_j^{(k)}$ is the impulse-response vector evaluated using STMs along $\mathbf{x}_{\text{nom}}^{(k)}(t)$. Substituting into the miss-distance penalty and absorbing the linear shift into an *effective* target gap

$$\mathbf{d}_{\text{eff}}^{(k)} = (\mathbf{x}_{\text{target}} - \mathbf{x}_{\text{nom}}^{(k)}(t_f)) + \sum_j q_j^{(k)} \mathbf{b}_j^{(k)},$$

yields a QUBO of the same algebraic form as (78) with $\mathbf{d} \rightarrow \mathbf{d}_{\text{eff}}^{(k)}$ and $\mathbf{b} \rightarrow \mathbf{b}^{(k)}$.

The outer loop is then:

1. Initialize $\mathbf{q}^{(0)} = \mathbf{0}$ (coast).
2. Build the QUBO linearized around $\mathbf{q}^{(k)}$.
3. Sample a candidate $\hat{\mathbf{q}}$ (any sampler: BF, SA, MILP, Leap Hybrid, QPU).
4. Propagate $\hat{\mathbf{q}}$ under the true nonlinear dynamics and measure the actual miss $\|\mathbf{x}(t_f) - \mathbf{x}_{\text{target}}\|$.
5. *Trust-region accept*: if the candidate's true miss decreases, accept $(\mathbf{q}^{(k+1)} \leftarrow \hat{\mathbf{q}})$; else terminate.
6. Stop when the schedule is a fixed point or the miss improvement falls below tolerance.

Step 5 is the key safeguard. The linearized model is only locally accurate; a candidate that the QUBO predicts to be better may in fact worsen the true miss because the linearization is loose. Rejecting non-improving candidates makes the loop monotone in the truth metric and guarantees that the returned schedule is no worse than coast — a property the single-pass solve does not have.

10.6 Multi-direction scheduling

The single-channel formulation restricts thrust to a single direction (e.g. tangential) at every step. In practice, a spacecraft may need to combine thrust components along different axes. The *multi-channel* extension introduces K independent thrust directions, each with its own on/off binary variable per step.

Variable structure. Let the thrust channels be labelled $c = 1, \dots, K$ (e.g. $K = 2$ for tangential and normal). At each decision step i , the QUBO contains one variable $q_{c,i} \in \{0, 1\}$ per channel. The total number of binary variables is $M = K \cdot N$. Variables are ordered channel-major:

$$\mathbf{q} = (\underbrace{q_{1,1}, \dots, q_{1,N}}_{\text{channel 1}}, \underbrace{q_{2,1}, \dots, q_{2,N}}_{\text{channel 2}}, \dots) \in \{0, 1\}^M.$$

Available directions. Four velocity-relative directions are supported:

- **tangential** — along $\mathbf{v}/\|\mathbf{v}\|$,
- **anti-tangential** — opposite to $\mathbf{v}$,
- **normal** — perpendicular to $\mathbf{v}$, rotated 90° counter-clockwise,
- **anti-normal** — opposite to normal.

These are evaluated at the coast-arc state at each decision point, so the direction naturally follows the spacecraft’s trajectory.

QUBO assembly. Each (channel, step) pair contributes an impulse response vector $\mathbf{b}_{c,i} = \Phi(t_i, t_f) \mathbf{B}_{\text{ctrl}} \mathbf{u}_{c,i} \Delta t$, where $\mathbf{u}_{c,i}$ is the thrust acceleration vector for channel c at step i . The QUBO objective generalizes naturally:

$$f(\mathbf{q}) = \left\| \sum_{c=1}^K \sum_{i=1}^N q_{c,i} \mathbf{b}_{c,i} - \mathbf{d} \right\|_{\mathbf{W}}^2 + \alpha \sum_{c,i} q_{c,i}. \quad (78)$$

The $M \times M$ QUBO matrix $\mathbf{Q}$ has the same structure as (74)–(76), with the summation indices running over all (c, i) pairs.

Mutual exclusion constraint. If the spacecraft has a single engine that cannot fire in two directions simultaneously, an exclusion penalty is added:

$$f_{\text{excl}}(\mathbf{q}) = P \sum_{i=1}^N \sum_{c < c'} q_{c,i} q_{c',i},$$

where P is a penalty weight chosen large enough to make simultaneous firing energetically unfavorable. This adds P to the off-diagonal QUBO entries coupling channels at the same step, and preserves the QUBO structure — no auxiliary variables are needed.

Scaling. With K channels and N steps, the QUBO has $M = KN$ variables. For $K = 2$ (tangential + normal), a 50-step problem requires 100 qubits — still well within hardware limits. The search space grows to 2^{KN} , making the combinatorial advantage of quantum annealing even more relevant than in the single-channel case.

11 Scheduling results

11.1 Validation: SA matches brute force

A cislunar transfer scenario was constructed with initial state near Earth, a target generated by continuous thrust propagation, and $N = 15$ decision steps. The QUBO was solved by both brute force and simulated annealing (1000 reads). SA recovered the exact brute-force optimum in every run, confirming that the QUBO energy landscape is well-conditioned for annealing-based samplers.

11.2 Scaling analysis

The critical motivation for quantum annealing is scaling. Table 1 reports solve times for brute force and SA as N increases.

Key observations:

- **Brute force** scales as $O(2^N)$: doubling time with each additional decision step. At $N = 20$ it takes ~ 10 s; at $N = 30$ the extrapolated time exceeds 10^4 s (~ 3 hours); at $N = 50$ it is astronomically large ($\sim 10^{15}$ evaluations).
- **Simulated annealing** grows subexponentially in this regime and matches the brute-force optimum for all $N \leq 20$. However, SA is a heuristic: there is no guarantee it will find the global minimum for larger N with strong couplings.
- **D-Wave QPU** anneal time is ~ 20 μ s regardless of N , as long as the QUBO fits on the hardware graph. A 50-step scheduling problem requires 50 qubits, well within the Advantage2 processor’s > 5000 -qubit capacity.

N	2^N	BF (ms)	SA (ms)	SA = BF?
8	256	3	133	Y
10	1 024	8	137	Y
12	4 096	30	148	Y
14	16 384	116	159	Y
16	65 536	435	181	Y
18	262 144	1 920	215	Y
20	1 048 576	10 100	249	Y

Table 1: Brute-force vs. simulated-annealing solve times for the on/off scheduling QUBO. SA matches the brute-force global optimum at every size tested. BF time doubles with each additional step ($O(2^N)$); SA grows slowly.

11.3 Where quantum advantage emerges

The regime $N \gtrsim 30$ is where quantum annealing becomes compelling:

- Classical brute force is intractable ($2^{30} > 10^9$).
- SA may find good solutions but lacks optimality guarantees.
- The QPU solves the *exact same* QUBO in ~ 20 μ s, with the annealing physics naturally exploring the combinatorial landscape.

This is a qualitative difference from the continuous-lstsq QUBO, where the QPU was solving a worse approximation of a problem that classical methods handle exactly. Here, the QPU solves the *exact* problem, and it does so faster.

11.4 Multi-direction results

Table 2 compares single-channel (tangential only) and multi-channel (tangential + normal) scheduling on a lateral-correction scenario with $N = 10$ decision steps and position-only targeting ($W = \text{diag}(1, 1, 0, 0)$). The spacecraft departs from a high Earth orbit ($r \approx 200\,000$ km) and the target is generated by propagating with pure normal thrust ($a = 0.02$, $t \in [0, 1]$), creating a gap that tangential burns alone cannot close effectively.

Mode	Qubits	Burns	Position miss	Improvement
Tangential only	10	1	1.02×10^{-2}	(baseline)
Tangential + Normal	20	9	2.61×10^{-5}	−99.7%
Exclusive (tan + nor)	20	9	8.18×10^{-5}	−99.2%

Table 2: Comparison of single- and multi-channel scheduling on a lateral-correction scenario. The multi-channel QUBO combines tangential and normal burns to reduce the position miss by nearly three orders of magnitude. The exclusive variant, which enforces at most one channel per step via a quadratic penalty, still achieves 99.2% improvement by alternating between thrust directions.

The multi-channel advantage emerges when the target requires corrections outside the velocity direction. The scenario uses position-only targeting, which is physically realistic for waypoint problems where velocity can be corrected by a subsequent maneuver. The key insight is that the STM couples all state components over long arcs, washing out the distinction between thrust directions. A shorter transfer arc ($t_f = 1$ vs. $t_f = 3$) preserves the directional separation and allows normal burns to retain their lateral character at the final time.

The multi-channel formulation scales naturally: With $K = 2$ channels and $N = 50$ steps, the QUBO has $M = 100$ binary variables — a 2^{100} search space that is utterly intractable by brute force but fits on 100 physical qubits.

11.5 Linearization-gap and iterative correction

The single-pass QUBO assumes the first-order STM expansion is accurate over the whole time interval. In a long-arc cislunar correction at $T = 4$ nondim ($N = 15$, $v_{\text{inj}}/v_{\text{circ}} = 1.187$) the assumption fails: the

QUBO’s optimum schedule, when propagated under the true nonlinear CR3BP, lands $\sim 143\,000$ km from the target, almost identical to coast and far from the linearised prediction. The iterative re-linearisation driver (Section 10.5) re-builds the QUBO around the current schedule at each outer iteration and applies a trust-region accept rule; on this scenario it converges in four outer iterations to a schedule whose true miss is $\sim 4\,800$ km, a $30\times$ reduction at the cost of one extra burn. Figure 2 compares the two trajectories side by side.

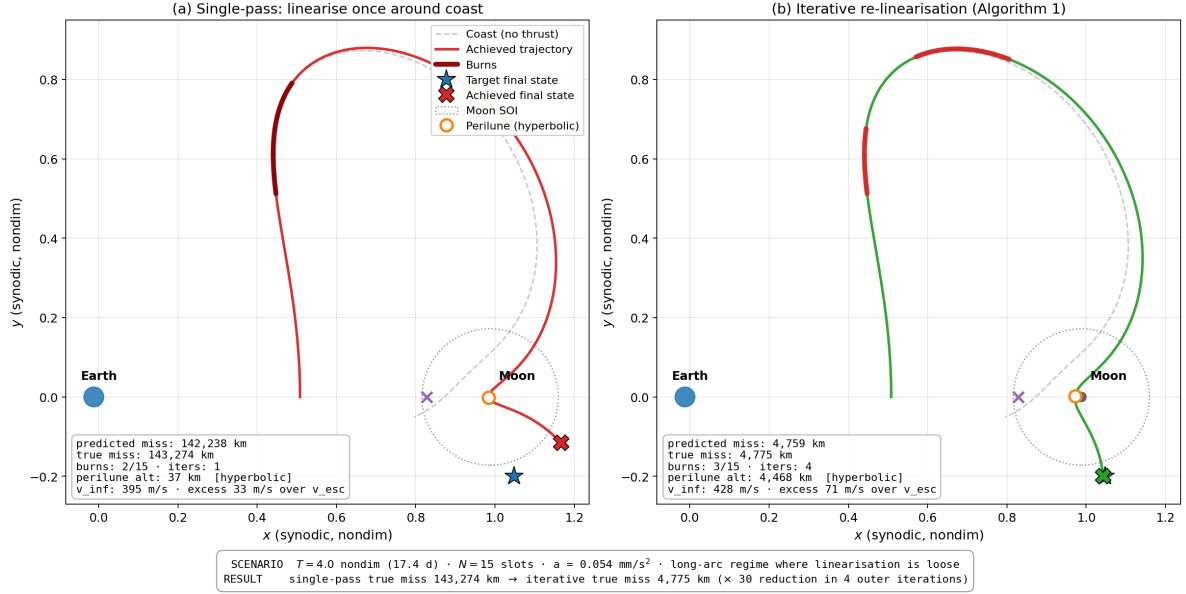


Figure 2: Single-pass QUBO (left panel) versus iterative re-linearisation (right panel) on a long-arc cislunar correction at $T = 4$ nondim. The blue star is the target final state; the cross is each schedule’s achieved final state. The single-pass scheduler picks two burns (red segments) whose linearised effect is correct but whose nonlinear effect leaves the spacecraft on a markedly different arc. The iterative scheduler converges in four outer iterations to a three-burn schedule that lands within 4800 km of the target.

For arcs longer than $T \approx 5$ nondim the linearisation is so loose that the first candidate is non-improving and the iterative loop terminates at coast — it *abstains* rather than acting on bad information. The single-pass solve, lacking the trust-region rule, would in this regime recommend a schedule that is worse than doing nothing.

11.6 Second application: lunar-orbit station-keeping

The same iterative scheduler applies to mission phases beyond injection correction. Figure 3 shows a 5000-km circular lunar orbit perturbed by a 0.5% tangential velocity error at t_0 . Without correction the spacecraft drifts $\sim 1\,300$ km from the nominal endpoint after two revolutions; the iterative QUBO selects a single anti-tangential channel and uses 4.4 m/s of Δv to reduce the position miss to ~ 530 km.

11.7 Classical MILP baseline

The MILP lift described in Section 10 provides a certified-optimal classical reference at small to medium N . Solved through HiGHS via `scipy.optimize.milp`, the lift produces an exact optimum at $N = 20$ in ~ 12 s with optimality certificate; at $N = 30$ the same solver cannot certify optimality within a 120-second budget but returns a feasible incumbent indistinguishable from SA. The McCormick lift creates $N(N-1)/2$ auxiliary binaries and a dense LP relaxation, so the classical-MILP scaling crosses the simulated-annealing curve in the $N \approx 25$ –30 region. This is the operationally relevant crossover: where SA dominates a state-of-the-art classical exact solver, the case for moving to D-Wave hardware becomes quantitatively defensible.

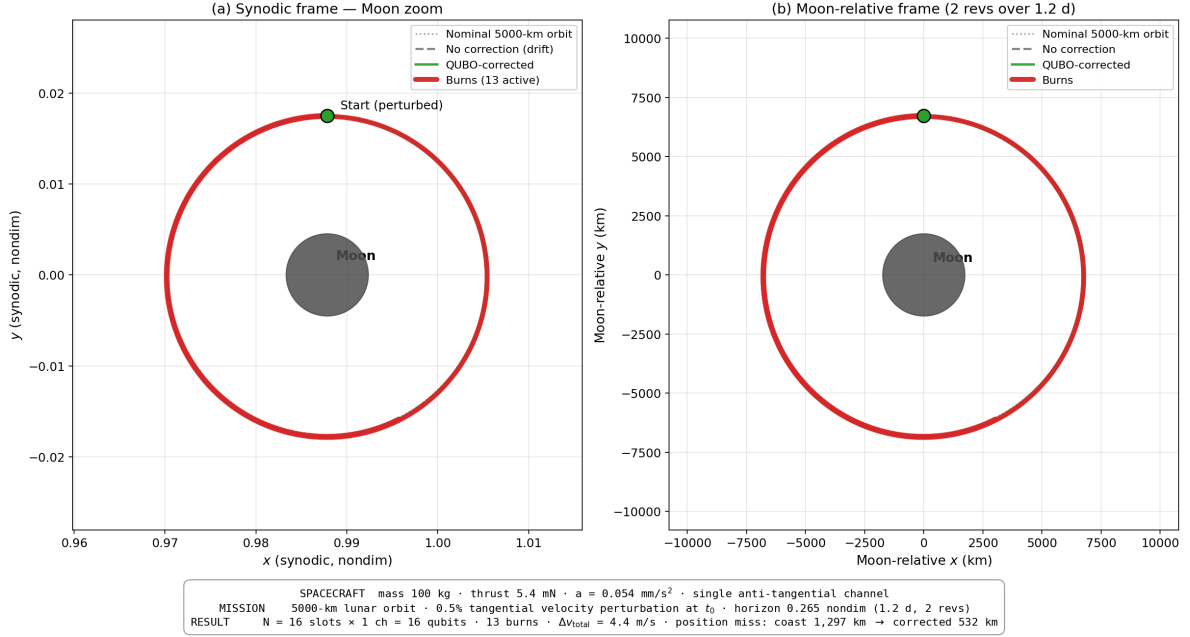


Figure 3: Lunar-orbit station-keeping with the iterative QUBO at the same spacecraft class as the injection-correction scenario (100-kg microsatellite, 5.4-mN ion thruster). Left: synodic frame zoomed near the Moon over two revolutions (~ 28 hours). Right: Moon-relative frame in km. The QUBO uses position-only target weighting ($W = \text{diag}(1, 1, 0, 0)$), so a phase-shift along the orbit affects the velocity vector but not the position objective — the right metric for a station-keeping mission that cares about position, not phase.

12 Multi-arc capture and end-to-end mission

The iterative re-linearisation of Section 10.5 closes the linearisation gap of a *single* arc. To complete the mission (spiral out from GEO, correct the cislunar transfer, and capture into a bound lunar orbit) we extend the scheduler in two ways: a sliding-window structure that chains short single-arc QUBOs across many lunar revolutions, and a composition-of-phases structure that switches the action set, target weighting and thrust regime as the spacecraft moves between Earth-, cislunar- and Moon-dominated regimes.

12.1 Sliding-window QUBO for lunar capture (Phase 3)

A representative case the single-arc QUBO cannot handle is lunar orbit insertion (LOI) from a hyperbolic flyby: the spacecraft passes the Moon once, must shed several tens of metres per second of excess velocity to bind its orbit, and then circularise over many subsequent revolutions. A single-arc QUBO covering this entire process would have a linearisation gap so large that no schedule survives the trust-region rule of Section 10.5. The implementation lives in `qalunar/qubo/lunar_capture.py` and is exposed as `solve_lunar_capture_sliding_window`.

Sliding-window structure. We split the mission into W short arcs of duration ΔT_w , each of which is small enough that the iterative re-linearisation converges. Between windows the spacecraft state is propagated under the true nonlinear CR3BP and becomes the initial state of the next window. The total schedule is the concatenation of per-window schedules.

Window-length cap. The natural choice for ΔT_w is a fraction of the local Moon-relative orbital period $T_{\text{loc}} = 2\pi\sqrt{r_w^3/\mu}$, with r_w the Moon-relative radius at the start of window w . This adapts the window to the local dynamics. However, T_{loc} diverges as r_w grows (loose orbits or escapes), and an unguarded fractional-period window would become arbitrarily long once the orbit is large – by which point the linearisation has long since collapsed. We therefore cap ΔT_w at a fixed ΔT_{max} so the linearisation regime stays valid even when the orbit is loose.

Action-radius gating. A second safeguard restricts QUBO scheduling to windows in which the spacecraft is within a threshold r_{act} of the Moon. Outside this radius, dynamics are dominated by Earth and the Moon-relative linear target is meaningless: every candidate fails the trust-region rule and the loop produces no progress while burning through windows. With gating, those windows become unforced drifts – the spacecraft is propagated freely under the CR3BP until it returns inside r_{act} , at which point QUBO scheduling resumes. Choosing r_{act} on the order of a few perilune radii confines the QUBO action to the region where anti-tangential thrust efficiently dissipates orbital energy.

Per-window target. For each window, we set the QUBO target to a state whose Moon-relative inertial velocity has magnitude $v_{\text{circ}}(r) = \sqrt{\mu/r}$ in the direction of the spacecraft’s current Moon-relative velocity, evaluated at the unforced end-of-window position $\mathbf{x}_{\text{eow}}$:

$$\mathbf{x}_{\text{target}}^{(w)} = [\mathbf{r}_{\text{eow}}, v_{\text{circ}}(\|\mathbf{r}_{\text{eow}} - \mathbf{r}_{\text{Moon}}\|) \hat{\mathbf{v}}_{\text{Moon-rel}}^{\text{eow}}] \quad (79)$$

expressed in the synodic frame after the standard $\boldsymbol{\omega} \times \mathbf{r}$ correction. Position is left unweighted ($W_{\text{pos}} = 0$) and only velocity components are weighted in the QUBO objective: the target says *stay where the unforced trajectory takes you, but slow to the local circular speed*. With a single anti-tangential channel, this target elicits firings precisely where the spacecraft is moving faster than circular – i.e. near perilune of an elliptical pass – which is the energy-dissipation regime for lunar capture and circularisation. At apolune the spacecraft is already slower than v_{circ} , anti-tangential thrust cannot speed it up, and the QUBO correctly stays off.

Termination and diagnostics. The outer loop terminates after W windows have been processed. We monitor the two-body energy $E_w = \frac{1}{2}\|\mathbf{v}_w^{\text{rel}}\|^2 - \mu/\|\mathbf{r}_w^{\text{rel}}\|$ and the Moon-relative apolune radius r_{apo} across windows. Captured states have $E_w < 0$ and finite r_{apo} ; loose-orbit windows have r_{apo} in the tens of thousands of kilometres; escaped or pre-capture states have $E_w \geq 0$ and $r_{\text{apo}} = \infty$.

Per-window guarantees. Each window is itself a call to the iterative driver of Section 10.5 with a short ΔT_w , so the trust-region monotonicity property applies window-by-window: the schedule produced for window w has true nonlinear miss no worse than coast *within that window*. The composition does not admit a global monotonicity guarantee in E_w because the unforced drift between windows can both raise or lower the energy under CR3BP perturbations, but in practice anti-tangential firings near perilune dominate and the energy decreases over the course of the schedule (see Section 12.2).

Sensitivity sweep. The three principal parameters (ΔT_{max} , r_{act} , brake factor on v_{circ}) trade off as expected: larger ΔT_{max} allows more burn slots per window but worsens the in-window linearisation; larger r_{act} allows scheduling further from the Moon at the cost of less effective burns; a brake factor below 1 targets a sub-circular speed and produces lower-apolune captures at higher Δv cost. The sweep is reproduced by `scripts/run_sliding_window_sweep.py` and reported in the companion paper.

12.2 End-to-end mission: composing Phases 1, 2 and 3

The end-to-end demo (`scripts/run_full_mission_demo.py` for numbers, `scripts/run_mission_figure.py` and `run_continuity_figure.py` for figures) flies a single 100 kg microsatellite with a single throttleable ion/Hall thruster (350 mN main, 5.4 mN cruise) from GEO to a bound near-circular lunar orbit, with no impulsive burns. The three phases share the same spacecraft and the same engine; only the throttling state, the action channel and the target weighting change between phases.

Phase 1 – Earth ascent + injection boost. The analytical Edelbaum spiral of Section 10 (implemented in `qalunar/reference/edelbaum.py`) lifts the spacecraft from GEO ($r \approx 42\,164$ km) to a circular parking orbit at $r = 200\,000$ km at *main* thrust. A continuous all-on tangential arc on the same thruster then adds the 264 ms^{-1} needed to inject onto a Moon-bound trajectory; this corresponds to 20.9 h of thrust and replaces what would otherwise be an impulsive chemical burn. Phase 1 totals $\Delta v \approx 1924\text{ m s}^{-1}$ over 6.4 d.

Phase 2 – cislunar correction. The thruster throttles to cruise so each binary slot delivers the $\sim 4.4 \text{ m s}^{-1}$ needed to discriminate trajectory corrections at the metres-per-second level. This phase is exactly the long-arc scenario validated in Section 11.5: $T = 4$ nondim, $N = 15$, iterative re-linearisation, $\Delta v \approx 16 \text{ m s}^{-1}$ in three burns over 17.4 d, leaving the spacecraft on a hyperbolic flyby with perilune at $\sim 4500 \text{ km}$ altitude.

Phase 3 – sliding-window lunar capture. The thruster returns to main power and the sliding-window scheduler of Section 12.1 runs from the raw Phase-2 perilune state – no impulsive pre-capture burn. With $\Delta T_{\max} = 0.01$ nondim ($\sim 1 \text{ h}$), $N = 12$ decision steps per window, an action radius of $3 r_{\text{perilune}}$, and a single anti-tangential channel, the loop fires during the close approach to the Moon. After ~ 200 windows the energy has dropped from $E = +0.087$ (hyperbolic) to $E = -0.38$ (bound) and the apolune has collapsed from infinity to $r_{\text{apo}} \approx 6212 \text{ km}$, a near-circular bound orbit at the perilune radius. Phase 3 expends $\Delta v \approx 561 \text{ m s}^{-1}$ in 8 d.

Mission totals. The complete mission delivers $\Delta v_{\text{total}} \approx 2502 \text{ m s}^{-1}$ in 31.8 d of flight, with every metre per second produced by binary on/off commands of a single throttleable thruster (Table 3). Phase 1 dominates the Δv budget at 77% of the total, Phase 3 is the second-largest cost at 22%, and Phase 2 contributes 0.6%, essentially free in comparison; yet without the Phase-2 correction the 22 000-km miss precludes any usable perilune passage and the mission cannot continue.

Table 3: All-binary Earth–Moon mission breakdown. Single 100 kg microsatellite with a throttleable ion/Hall thruster (350 mN main / 5.4 mN cruise). No impulsive burns.

Phase	Δv (m/s)	TOF (days)	Mode
1: Edelbaum spiral + injection boost	1924	6.4	main, tangential
2: cislunar correction (QUBO)	16	17.4	cruise, tangential
3: sliding-window QUBO capture	561	8.0	main, anti-tangential
Total	2502	31.8	all binary

Why throttling is necessary. Bumping every phase to main thrust breaks Phase 2: at 350 mN each binary slot delivers $\sim 350 \text{ m s}^{-1}$, vastly more than the metres-per-second resolution needed to discriminate good trajectory corrections. The QUBO either fires zero slots (under-correction) or a single slot (overshooting by an order of magnitude). Throttling to 5.4 mN restores the fine-grained regime that Phase 2 requires.

Why the action-radius gate is necessary. Without gating, the sliding-window solver fires the QUBO during windows in which the spacecraft is close to L1 and dominated by Earth’s gravity. The Moon-relative circular target has no physical meaning in that regime; the trust-region rule rejects every candidate, but more dangerously, when a candidate is accidentally accepted (because the linearised cost happens to look better) the true CR3BP propagation perturbs the orbit further. With the gate, the spacecraft drifts naturally between perilune passages and QUBO firings concentrate where they are physically effective.

12.3 Connection to primer-vector theory

The on/off scheduling QUBO never computes a Pontryagin costate — the decision variables are the binary controls themselves, not the adjoint. Once a schedule has converged under the iterative driver, however, a costate can be reconstructed *a posteriori* along the resulting nonlinear trajectory by integrating $\dot{\lambda} = -A(\mathbf{x}(t))^T \lambda$ backward in time from the terminal transversality condition $\lambda(t_f) = 2\mathbf{W}(\mathbf{x}_f - \mathbf{x}_{\text{target}})$. The tangential primer-vector analogue is the scalar switching function

$$S(t) = -\lambda_v(t) \cdot \hat{\mathbf{v}}(t), \quad (80)$$

where λ_v is the velocity sub-vector of the costate and $\hat{\mathbf{v}}$ the unit velocity vector. Pontryagin’s minimum principle predicts that the optimal tangential bang-bang control fires the engine when $S > 0$ and coasts otherwise. Comparing this continuous-time prediction with the QUBO’s discrete schedule

provides a sanity check on the bridge between the binary formulation and indirect primer-vector methods [9]. The implementation is in `qalunar/reference/primer_vector.py` and the figure is produced by `scripts/run_primer_vector_figure.py`.

Applied to the converged iterative schedule of Section 11.5 ($T = 4$ nondim, $N = 15$, three ON slots out of fifteen), the reconstruction yields a switching function whose support is a contiguous interval near the second half of the arc. Three observations stand out:

1. Every QUBO ON-slot has its slot centre inside the PMP-predicted ON region ($S(t_i) > 0$, $3/3$); the converse pathology — a QUBO burn placed where PMP predicts coast — does not occur.
2. The QUBO does not saturate the $S > 0$ interval as a continuous PMP would in the unconstrained case: nine of the fifteen slot centres lie in the $S > 0$ region but only three are activated. With no fuel penalty, continuous PMP would fire the engine across the entire $S > 0$ interval; the binary QUBO instead selects a minimal subset of those slots sufficient to land the target within tolerance, because the squared-norm objective is minimised by the smallest set of impulse-response vectors $\mathbf{b}_i$ whose sum cancels the gap $\mathbf{d}_{\text{eff}}$, not by activating every slot whose individual contribution would align with $-\mathbf{d}_{\text{eff}}$.
3. The costate exhibits strong backward amplification near t_0 , a direct manifestation of CR3BP sensitivity to the initial state: small perturbations early in the arc compound through the chaotic flow and produce large terminal-state changes, which is exactly the regime where the linearisation gap is widest — and exactly why the iterative loop is required to make the QUBO faithful in the first place.

This reconstruction is a self-consistency check: $\boldsymbol{\lambda}$ is built from the QUBO’s own final state and propagated through the QUBO’s own trajectory. A fully external comparison would solve the same problem via indirect methods (the route taken in [9] via homotopy continuation in ρ and u_{max}) and overlay the two switching functions. We leave that comparison to future work, as the in-house direct-collocation NLP referenced earlier is energy-optimal rather than fuel-optimal and therefore lacks a directly comparable primer vector.

12.4 Comparison with state-of-the-art Earth–Moon transfers

Recent work by Taylor et al. [9] from the Purdue Orbital student team designs Earth-to-NRHO and Earth-to-LLO transfers via Pontryagin’s minimum principle and primer-vector theory, with hyperbolic-tangent smoothing and homotopy continuation in ρ and u_{max} to converge on the impulsive limit. Their reported totals are 3.93 km s^{-1} in 5.2 d for LEO $\rightarrow$ NRHO and 3.91 km s^{-1} in 4.7 d for LEO $\rightarrow$ LLO. Table 4 places those numbers next to the qalunar end-to-end binary-QUBO result.

Table 4: End-to-end Earth–Moon transfer comparison. The *method* column lists the principal optimisation framework; totals are not directly comparable due to differences in starting orbit, propulsion system and final orbit (see text).

Source	Transfer	Δv (km/s)	TOF (d)	Method
This work	GEO $\rightarrow$ bound lunar orbit	2.50	31.8	binary QUBO, low-thrust ion/Hall
[9]	LEO $\rightarrow$ 9:2 NRHO	3.93	5.2	PMP + primer vector, impulsive chemical
[9]	LEO $\rightarrow$ LLO (100 km)	3.91	4.7	PMP + primer vector, impulsive chemical

The comparison is not apples-to-apples in three respects: (i) the starting orbit differs (GEO vs. LEO — a $\sim 3.9 \text{ km s}^{-1}$ Hohmann gap that favours qalunar); (ii) the propulsion system differs (low-thrust electric vs. impulsive chemical — low thrust buys Δv efficiency at the cost of TOF); and (iii) the final orbit differs (qalunar reaches a near-circular bound orbit at the perilune radius rather than 100 km LLO or the 9:2 NRHO). Subject to those caveats, the qualitative conclusion is that the binary-QUBO formulation produces a Δv budget consistent with the indirect-method state of the art at the cost of an order of magnitude longer TOF — the expected low-thrust trade-off. The methodological novelty is not a Δv improvement but the demonstration that an entire Earth–Moon mission can be flown by binary on/off commands of a single throttleable thruster, with the QUBO scheduler and its sliding-window extension as the discrete-side analogue of the indirect primer-vector switching function [9].

13 Discussion: why the project changed direction

The original project plan, following Carbone et al. [1], aimed to extend the QUBO-encoded continuous least-squares approach from the two-body HCW setting to the nonlinear CR3BP. The investigation described in Section 9 revealed a fundamental mismatch between the tool (binary optimization) and the problem (continuous optimization):

1. **Classical methods solve the inner problem exactly.** Least squares finds the continuous minimum in ~ 5 ms. The QUBO *approximates* the same answer with quantization error, in far more time.
2. **Quantization error is a hard floor.** With p bits per variable, the best the QUBO can achieve differs from the true lstsq solution by $O(2^{-p})$ per component. This error propagates through the sequential linearization and prevents convergence below a quantization-dominated residual plateau.
3. **There is no asymptotic advantage.** Increasing p shrinks the error but linearly increases the number of qubits. The QUBO is always solving a harder version of a problem that classical methods solve trivially.

The pivot to on/off thrust scheduling resolves all three issues. The decision variables $q_i \in \{0, 1\}$ are *inherently binary*: there is no encoding, no quantization, no approximation beyond the standard STM linearization. The 2^N search space is genuinely hard for classical exhaustive methods, and quantum annealing is designed precisely for this class of combinatorial optimization.

The previous work (TFC+ELM transcription, lstsq-to-QUBO pipeline, sequential linearization) remains valuable as a reference implementation and a documented negative result: it shows where QUBO encoding of continuous problems breaks down and why naturally binary formulations are the scientifically sound path forward.

14 Glossary of symbols

Symbol	Meaning	Notes
μ	CR3BP mass parameter	$m_2/(m_1 + m_2)$; Earth–Moon ≈ 0.01215
r_1, r_2	Distances to primaries	r_1 from Earth, r_2 from Moon
Ω	Pseudo-potential	Centrifugal + gravitational + bookkeeping
Ω_x, Ω_y	Gradient of Ω	Drives position dynamics
$\Omega_{xx}, \Omega_{xy}, \Omega_{yy}$	Hessian of Ω	Time-varying along nominal
C_J	Jacobi integral	$2\Omega - \ \mathbf{v}\ ^2$
$(L_1, \dots, L_5)$	Lagrange points	L_4, L_5 stable for Earth–Moon
$\mathbf{r}, \mathbf{v}$	State position and velocity	$\mathbf{r} = (x, y)$, $\mathbf{v} = (v_x, v_y)$
$\mathbf{u}$	Thrust acceleration control	(u_x, u_y)
$\boldsymbol{\lambda}_r, \boldsymbol{\lambda}_v$	Costates	$\boldsymbol{\lambda}_r$ conjugate to $\mathbf{r}$, $\boldsymbol{\lambda}_v$ to $\mathbf{v}$
$\mathcal{H}$	Control Hamiltonian	$\mathbf{u}^* = -\boldsymbol{\lambda}_v$
T	Time of flight	Fixed
$(\mathbf{r}_0, \mathbf{v}_0)$	Initial BC	LEO state
$(\mathbf{r}_f, \mathbf{v}_f)$	Final BC	LLO state
$z \in [-1, 1]$	Normalized time	$z = c(t - t_0) - 1$, $c = 2/T$
$\omega_1, \dots, \omega_4$	Hermite switching functions	TFC BC-annihilators
$\mathbf{H}$	ELM basis matrix	$n \times L$, $H_{ij} = \tanh(w_j z_i + b_j)$
$\mathbf{F}$	Constrained expression matrix	ELM minus switching-projections
n	Training points per coordinate	Typical 40
L	ELM neurons per coordinate	Typical 20–80
$\boldsymbol{\xi}$	ELM output weights	Free parameters
$\mathbf{A}, \mathbf{B}$	Linear residual system	Shape $6n \times 6L$ and $6n$
λ_{tik}	Tikhonov strength	$\varepsilon_{\text{tik}} \sigma_{\max}(\mathbf{A})^2$
r	Effective rank after SVD truncation	Much smaller than $6L$ in practice
$\mathbf{V}_r$	Right basis of truncated SVD	$6L \times r$
$\boldsymbol{\alpha}$	Reduced coordinates	$\boldsymbol{\xi} = \mathbf{V}_r \boldsymbol{\alpha}$
p	Bits per reduced variable	Typical 8
$\mathbf{q} \in \{0, 1\}^{rp}$	QUBO bitstring	Physical qubits (1stsq encoding)
$\mathbf{Q}, \boldsymbol{\ell}, c$	QUBO triple	$E(\mathbf{q}) = \mathbf{q}^\top \mathbf{Q} \mathbf{q} + \boldsymbol{\ell}^\top \mathbf{q} + c$
$q_i \in \{0, 1\}$	Thrust on/off decision	$q_i = 1$: burn at step i
N	Number of decision steps	= number of qubits (scheduling)
$\Phi(t_i, t_f)$	State transition matrix	Maps perturbation at t_i to t_f
$\mathbf{B}_{\text{ctrl}}$	Control influence matrix	(4×2) , maps thrust to state derivative
$\mathbf{b}_i$	Impulse response vector	$\Phi(t_i, t_f) \mathbf{B}_{\text{ctrl}} \mathbf{u}_i \Delta t$
$\mathbf{d}$	Target state gap	$\mathbf{x}_{\text{target}} - \mathbf{x}_{\text{coast}}(t_f)$
α	Fuel penalty weight	Trades miss distance vs. number of burns
$\mathbf{W}$	Target weighting matrix	$\text{diag}(w_1, w_2, w_3, w_4)$; default $\mathbf{I}$

Bibliography

References

- [1] De Grossi, F., Carbone, A., Spiller, D., Ottaviani, D., Mengoni, R., Circi, C., *Transcription and optimization of an interplanetary trajectory through quantum annealing*, *Astrodynamics* **9**:195–215, 2025.
- [2] Szebehely, V., *Theory of Orbits: The Restricted Problem of Three Bodies*, Academic Press, 1967.
- [3] Koon, W., Lo, M., Marsden, J., Ross, S., *Dynamical Systems, the Three-Body Problem and Space Mission Design*, 2011.
- [4] Mortari, D., *The Theory of Connections: Connecting Points*, *Mathematics* **5**(4):57, 2017.

- [5] Huang, G.-B., Zhu, Q.-Y., Siew, C.-K., *Extreme learning machine: Theory and applications*, Neuro-computing **70**:489–501, 2006.
- [6] Kelly, M., *An Introduction to Trajectory Optimization: How to Do Your Own Direct Collocation*, SIAM Review **59**(4):849–904, 2017.
- [7] Betts, J. T., *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*, 2nd ed., SIAM, 2010.
- [8] Conn, A. R., Gould, N. I. M., Toint, P. L., *Trust-Region Methods*, MPS-SIAM Series on Optimization, SIAM, 2000.
- [9] Taylor, T. E., Madden, K. F., Bohne, S. L., Vetter, M. J., *An Earth–Moon Trajectory and Mission Design*, Purdue Orbital, Purdue University, West Lafayette, IN, 2026.